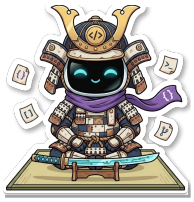


Anti-Vibe Coding Standard

Continuous Repository Alignment for Proof-Carrying AI Pull Requests

Find the vibe. Prove the merge. Repair the repo.

Jankurai Project • github.com/jeppsontaylor/jankurai

Status `public_draft` **Standard** `0.7.0` **Auditor** `0.7.0`
Schema `1.5.0` **Paper edition** `2026.05-ed7`

Abstract

AI-assisted coding makes plausible code cheap and vibe-code drift expensive. Traditional technical debt becomes *evidence debt* when a repository cannot prove who owns a changed path, which generated boundaries apply, which behavior changed, or which proof makes a refactor safe. Jankurai is a versioned repository conformance standard for finding and reducing *vibe artifacts*: ownerless paths, unmapped proof, hand-edited generated zones, stale contracts, false-green tests, missing security evidence, overbroad agent permissions, unproven UI changes, stale waivers, and unreceipted review claims. In this paper, *proof* means repository-local evidence receipts, not a formal proof of full program semantics.

Jankurai treats the repository as the alignment layer for human- and agent-authored code. A local, pull-request, and CI loop maps each changed path to bounded authority, proof lanes, evidence receipts, repair queues, or expiring waivers. Its central artifact is the *merge witness*, a versioned binding among changed paths, owner routes, required proof receipts, observed evidence, score deltas, missing-evidence decisions, tool identity, artifact digests, and commit identity. The result is not a larger prompt or a hosted trust service; it is a reproducible merge decision that can be audited at the current commit.

The paper contributes a stack-neutral repository model for continuous repository alignment with owner maps, test maps, generated-zone policy, stable rule identifiers, version manifests, bounded authority, proof receipts, audit reports, repair queues, merge witnesses, and a seed conformance suite. It also frames *agent-first repository design*: code and policy shaped so agents can find owners, avoid generated zones, select one proof lane, receive stable errors, repair narrow scope, and leave receipts. The claim is bounded: Jankurai does not prove full program semantics, does not require a Rust/TypeScript/PostgreSQL stack, and does not treat scores as merge approval. It proposes a version-controlled standard for turning vibe coding into repairable evidence.

Index Terms—vibe coding, vibe artifacts, technical debt, evidence debt, refactoring, agent-first repository design, continuous repository alignment, proof-carrying pull requests, merge witness, repair queues, agent efficiency, repository governance, software evidence.

I. From Language Chaos to Verified Merge

Software history is often presented as a clean ladder of progress: assembly to C, C to managed runtimes, dynamic languages to typed product stacks, and now AI-assisted programming. The lived history is messier. Programming languages are human inventions shaped by fashion, hardware pressure, academic taste, corporate platforms, tooling accidents, hiring markets, and the local pain of the decade. Each new language promises to compress some burden. Each also creates a new dialect, package culture, build system, error style, documentation habit, and set of failure modes that humans must learn before they can safely change a system.

The early languages compressed machine pain. Assembly gave names to opcodes and registers. Backus's Turing Award lecture framed the deeper trap as the "von Neumann style": languages and machines reinforcing each other's assignment-and-memory habits [1]. C made portable systems programming possible, as Ritchie's history of C makes clear, but it also made memory safety and low-level execution semantics permanent review burdens [2]. Managed runtimes compressed memory management and deployment portability into VM policy. Scripting languages compressed iteration time and glue code, but often moved correctness from compiler feedback into convention, test discipline, and senior reviewer memory. The web multiplied the problem: JavaScript became the universal runtime by distribution, not by design, and TypeScript later compressed some of that chaos into a typed product surface [3]. GitHub's 2025 Octoverse report describes TypeScript as the most-used language on GitHub and AI as mainstream in development [4]; that is not an accident. The dominant language is now entangled with the dominant collaboration substrate.

The deepest advances were not only new syntax. Dijkstra's attack on unstructured control flow, Parnas's information-hiding decomposition criteria, and McCabe's complexity measure all tried to make local reasoning cheaper [5]–[7]. Conway's law warned that software structure also follows communication structure, so the codebase records organization as well as design [8]. Those ideas still matter in agent-assisted work: a repository that hides ownership, boundary policy, and proof routes is asking the next author to reconstruct the design theory from residue.

There were also hopeful branches that did not become universal defaults. Julia attacked the two-language problem in scientific computing with multiple dispatch and JIT performance [9]. Elm showed how frontend reliability can improve when impossible states are harder to express. F#, Clojure, Haskell, Scala, Elixir, Nim, Crystal, D, and other ecosystems carried serious ideas about type systems, concurrency, supervision, native compilation, and expressiveness. Many survived as strong niches. Many failed to become default product infrastructure. The usual cause was not that the ideas were stupid. It was standard gravity: package maturity, deployment norms, hiring supply, build speed, debugging habits, cloud operations, security scanning, database migration practice, interop, and the amount of public example code that humans and agents can learn from.

TABLE I
Bottleneck shift for agent-era repositories.

Era	Primary bottleneck
Old bottleneck	Syntax, machine constraints, memory layout, and local comprehension.
Current bottleneck	Merge evidence, owner routing, proof freshness, generated-boundary integrity, and repair locality.

This is the old human bottleneck. Developers do not merely learn syntax; they learn a language's social contract. Where does business truth live? Which errors are expected and which are exceptional? Which generated files are sacred? Which tests prove behavior and which only prove the mock? Which layer owns authorization? Which migration is safe? Which dependency is normal? Which code is legacy and which code is a pattern? Brooks's "No Silver Bullet" warned that the essential difficulty of software lives in the conceptual structure, not only the notation [10]. In a large repository, that conceptual structure is rarely in one place. It is spread across conventions, old pull requests, tribal warnings, CI jobs, stale docs, and reviewers who remember why the last rewrite failed.

Technical debt names the same pressure over time. Cunningham's debt metaphor, Parnas's software-aging account, Lehman's evolution laws, and later technical-debt theory all describe how expedient decisions accumulate interest when the system keeps changing [11]–[15]. Refactoring is the repair promise, but empirical refactoring work shows that real teams refactor opportunistically and under imperfect knowledge, not as clean textbook phases [16], [17]. In an AI-assisted repository, debt interest includes evidence interest: every hidden convention, missing owner route, vague test claim, and undocumented waiver raises the cost of the next safe change.

AI changes the economics without removing the mess. Languages reduced notation, runtime, deployment, and local reasoning burdens; AI compresses the next layer, the production of plausible edits. A model can imitate syntax, follow local style, and generate code in almost any mainstream language. That makes first drafts cheap. It does not make repository intent explicit, prove ownership, or bind a patch to merge evidence. It can even make hidden ambiguity more expensive, because the model confidently fills gaps that human teams used to negotiate slowly. DORA's 2025 research frames AI as an amplifier of existing organizational capability rather than an automatic productivity guarantee [18]. Security evidence is less forgiving: Veracode reports syntactically plausible generated code with known flaws [19], and GitGuardian reports accelerating secret exposure in public GitHub activity, including AI-assisted commits [20], [21].

The new bottleneck is not language expression. It is merge evidence. An agent opens a pull request that changes a checkout flow. The diff touches product UI, a generated API client, a schema-adjacent file, tests, and documentation. The hard reviewer question is no longer whether the patch looks plausible. It is: what evidence makes this safe to merge?

That question is the center of Jankurai. A vibe artifact is not

merely bad code. It is repository residue that makes a change plausible but not auditable: an ownerless path, unmapped proof lane, hand-edited generated output, false-green test, missing security receipt, overbroad tool permission, unproven UI change, stale waiver, or review claim with no receipt. Vibe artifacts are the agent-era form of language chaos. They appear when the repository cannot say what owns the behavior, which proof applies, or what evidence would make the merge trustworthy.

The correct response is not to slow agents down until the old review process survives. The correct response is to redesign the repository as the alignment layer. The repository, not the prompt or reviewer memory, must encode ownership, allowed edit scope, proof lanes, generated boundaries, permission limits, evidence receipts, repair queues, and versioned conformance. Jankurai's operating claims are deliberately concrete: the repository is the alignment layer for human- and agent-authored code; vibe artifacts are detectable as repository states; findings are reducible when they route to owners, proof lanes, receipts, and repair queues; and agent efficiency improves when broad context rereads are replaced by path-scoped policy and reusable evidence.

The checkout-flow pull request is the running example. If the generated client changed by hand, the repository should emit `HLT-002-GENERATED-MUTATION`. If the UI changed without screenshot, ARIA, trace, or geometry evidence, it should emit `HLT-013-RENDERED-UX-GAP`. If the path has no owner or proof route, it should emit `HLT-003-OWNERLESS-PATH` or `HLT-004-UNMAPPED-PROOF`. The merge witness then records whether the PR can pass, needs human review, must block, violates a ratchet, or fails release gates.

Agent-native engineering is not "humans approve whatever the model says." It is human-directed engineering in which intent becomes bounded authority, changed paths become proof lanes, proof lanes leave evidence, evidence drives repair or waiver expiry, and repeated repairs become reusable primitives. The same machinery must govern human-authored code, because a vague PR from a person and a vague patch from an agent fail in the same place: the repository cannot tell what owns the behavior, which proof applies, or what evidence would make the merge trustworthy.

This paper makes four contributions. First, it defines a stack-neutral repository conformance model for continuous repository alignment. Second, it specifies a merge-witness artifact that binds changed paths to ownership, proof, evidence, and decision. Third, it turns missing evidence into stable rule IDs and an ordered repair queue. Fourth, it reports current seed conformance-suite evidence and separates the non-normative reference profile from Jankurai Core.

The language story matters because it explains why another linter, another benchmark, or another prompt convention is not enough. Human teams already spent decades trying to make software safer by choosing better languages. That helped, but it never eliminated the repository-level questions. A Rust module can still hide the wrong owner. A TypeScript component can still ship a broken checkout state. A Python service can still own truth it should not own. A generated client can still be edited by hand. A CI suite can still be green without proving the changed behavior. The next professional baseline therefore cannot be only a language choice. It has to be a repository interface.

Jankurai is that interface. It does not ask every team to adopt the same stack or abandon existing tools. It asks every conformant repository to expose the evidence that serious review already needs: changed paths, owners, generated boundaries, required proof lanes, current receipts, missing evidence, artifact digests, waivers, repair actions, and a closed merge decision. The claim is simple but demanding: if a repository cannot explain why a change is safe to merge, the patch is still a vibe, no matter who or what wrote it.

change proposal → changed paths → vibe artifact scan → owner route → proof lane → receipt → merge witness → pass/review/block/ratchet_fail/release_fail → repair queue or waiver → reduced baseline

Fig. 1. The Jankurai continuous repository alignment loop. A change is not trusted because it looks plausible; it is trusted only when changed paths, owners, proof receipts, missing evidence, artifact digests, and commit/tool identity are bound into a merge witness.

II. Running Example: Checkout PR

The checkout-flow pull request is intentionally ordinary. It changes a page a customer can see, a generated client the page imports, and the API contract that should own the generated shape. Jankurai treats that combination as evidence work, not as a request for reviewer intuition.

The example is also a boundary for claims in this paper. The current seed conformance lane validates fixture inventory and expected JSON shape. Unless a real per-fixture runner emits observed witness decisions, this edition reports expected checkout-style decisions as conformance expectations, not as completed empirical benchmark results.

III. Definitions and Threat Model

Vibe coding is any development mode where plausible code is accepted without explicit ownership, proof routing, generated-boundary checks, security gates, UX evidence, and repair evidence. Jankurai does not merely criticize vibe coding. It serializes vibe artifacts, assigns stable rule IDs, routes them to proof lanes, and reduces them through repair receipts.

The standard does not claim every repository using AI must adopt Jankurai. It claims a narrower and testable thing: a repository claiming Jankurai conformance must expose enough machine-readable structure for the claim to be audited. Conformance is a public interface, not a vibe.

TABLE II
Running checkout example from blocked witness to pass shape.

Stage	Jankurai output
Changed paths	`apps/web/src/checkout/Checkout.tsx`; `apps/web/src/generated/client.ts`; `contracts/openapi/api.yaml`.
Initial scan	Generated-zone touch on `client.ts`; UI surface changed; contract-adjacent source changed.
Findings	`HLT-002-GENERATED-MUTATION` if the generated client changed without regeneration proof; `HLT-013-RENDERED-UX-GAP` if the checkout surface lacks rendered evidence.
Required lanes	`web`, `contract`, and `security` from `agent/test-map.json`; owner route `web` plus `contracts`.
Missing receipts	Contract regeneration receipt, rendered UX receipt, screenshot/ARIA/geometry evidence, and digest-bound audit report when absent.
Blocked witness	`decision=block`; missing evidence includes `contract_receipt` and `rendered_ux_receipt`; repair queue names source contract, generator command, owner, and rerun lanes.
Repair	Edit the OpenAPI source if the API changed, regenerate the client, run contract proof, run rendered UX proof for checkout, attach redacted artifacts.
Final pass shape	Present receipts cover all required lanes; generated-zone touch has source proof; UX evidence is redacted and digest-bound; `decision=pass`.

TABLE III
CORE TERMS.

Term	Operational meaning
Vibe artifact	Repository residue that makes a change plausible but not auditable: ownerless paths, unmapped proof, hand-edited generated zones, stale contracts, false-green tests, missing security evidence, overbroad agent authority, unproven UI changes, stale waivers, or review claims without receipts.
Vibe-code drift	The measurable gap between claimed engineering intent and evidence attached to a change.
Continuous repository alignment	The practice of keeping ownership, edit scope, proof lanes, generated boundaries, permissions, evidence, repair queues, and conformance metadata synchronized with each merge.
Evidence debt	Missing or stale merge evidence that accumulates like technical debt: absent owner routes, unmapped proof lanes, unvalidated generated boundaries, undocumented waivers, or receipts that cannot prove the current change.
Review subjectivity gap	Reliance on taste, memory, seniority, or local convention where a rule ID, owner route, proof lane, receipt, or explicit waiver should exist.
Real-time repository conformance	A repeatable local/PR/CI loop in which each changed path is evaluated against current repository policy before merge; it is not a claim of an omniscient daemon.
Proof freshness	A receipt is current for the changed-path scope, commit, tool identity, command identity, and declared lane it claims to prove.
Receipt validity	A receipt is replayable or inspectable enough to bind command, cwd, inputs, tool version, result, artifacts, and digest to the current merge witness.
Policy authority	The ordered source of truth used to decide which instructions, maps, lanes, and adapter claims are trusted.
Adapter drift	Divergence between provider instruction files and canonical repository policy.
Evidence bundle	A versioned group of reports, receipts, logs, screenshots, ARIA snapshots, digests, and witness artifacts used to audit a claim.
Conformance claim	A machine-readable statement that a repository meets a named Jankurai level at a commit under a standard, auditor, and schema version.
Proof receipt	A durable record of a proof command, tool identity, inputs, result, and artifact path.
Merge witness	A versioned artifact binding changed paths, owner route, proof receipts, score delta, missing-evidence decision, artifact digests, commit identity, and tool version.
Generated zone	An output path repaired by changing the declared source contract or generator and regenerating, not by hand edit.
Repair queue	Ordered findings with rule IDs, owners, proof lanes, rerun commands, evidence requirements, and bounded next actions.
Reference profile	A non-normative stack or implementation profile that can satisfy the standard without defining it.

TABLE IV
THREAT MODEL FOR AUDITABLE MERGE.

Threat	Risk	Jankurai control
False plausibility	Agent or human patch looks idiomatic but violates architecture, data, security, or runtime expectations.	Stable rule IDs, owner maps, proof lanes, generated-zone policy, and merge witness.
Stale or copied claims	A stale artifact is attached to a commit, path scope, tool, or command that did not run.	Freshness checks, artifact digests, validity rules, and witness binding.
Adapter drift	Tool-specific instruction files widen permissions or contradict canonical policy.	Source-of-authority hierarchy, adapter verification, policy digests, and generated mirrors.
Flaky proof lanes	Intermittent tests hide real failures or make proof receipts unreliable.	Lane contracts with timeout, environment, retry policy, artifact paths, and failure classification.
Emergency bypass	Urgent work merges without owner, security, release, or proof evidence.	Expiring waivers, compensating controls, release-fail decisions, and repair receipts.
Evidence leakage	Screenshots, logs, traces, prompts, or artifacts expose secrets or customer data.	Redaction policy, artifact allowlists, secret scans, retention bounds, and privacy receipts.
CI downgrade	A workflow disables proof, widens permissions, removes artifacts, or changes branch gates.	CI hardening rule, workflow diff checks, pinned actions, and required evidence uploads.
Malicious MCP or tool schema	External tool output or schema attempts to change trusted policy or execute unsafe commands.	Untrusted-content isolation, schema validation, permission receipts, and no direct execution of generated text.
Generated test self-confirmation	Generated code and generated tests confirm the same wrong assumption.	Owner-owned acceptance criteria, negative fixtures, semantic assertions, and changed-path routing.
Subjective or rubber-stamp review	Review approval relies on taste, memory, or plausibility instead of reproducible evidence.	Stable rule IDs, human-review receipt rule, receipt requirement, explicit waiver or review receipt, and merge witness decision record.
Screenshots or logs leaking secrets	Browser and observability evidence captures tokens, PII, prompt text, or private data.	UX/security evidence redaction, hash-based references, and artifact privacy status.
Out of scope	Full semantic correctness, malicious compiler/runtime integrity, and organizational governance outside the repository.	Explicit limitations, signed-receipt future work, and no claim of total program proof.

IV. JANKURAI CORE STANDARD AND CONFORMANCE

Jankurai Core is the standard interface. It is not a claim that all repositories should use one language, database, CI provider, or coding agent. It binds prose, machine-readable maps, generated-zone policy, audit output, proof receipts, repair evidence, CI behavior, and release governance into repository-local conformance claims.

Conformance predicate. Repository R at commit C conforms to Jankurai Core level L under standard version S, auditor version A, and schema version K iff R can emit schema-valid, current artifacts showing that every required ownership, generated-zone, proof-lane, security, waiver, repair, and witness predicate for L is satisfied at C.

The standard's public primitive is the merge witness:

TABLE V
NORMATIVE BOUNDARY OF THE JANKURAI ECOSYSTEM.

Layer	Status
Jankurai Core	Stack-neutral normative conformance model: levels, decision enum, artifact validity, rule IDs, and witness semantics.
Reference auditor	Current Rust CLI implementation; useful evidence but not the only possible conformant implementation.
Schema bundle	Versioned JSON schemas for reports, proof receipts, repair queues, merge witnesses, and manifests.
Rule registry	Stable HLT rule families with required evidence, detector maturity, and repair metadata.
Reference profile	Non-normative stack/profile guidance. A conformant alternate profile must not be rejected because it differs from the reference stack.

$$\begin{aligned}\text{merge witness} = {} & \text{changed paths} + \text{owner route} \\ & + \text{required proof receipts} + \text{score delta} \\ & + \text{missing evidence decision} + \text{artifact digests} \\ & + \text{commit/tool identity}\end{aligned}$$

The machine field is `schema_version`. A schema bundle is prose for the collection of report, witness, receipt, and evidence schemas identified by that version. Emitted audit, report, and profile claims use `target_stack_id`. The current manifest still carries `target_stack` as the repository's profile label; this edition treats that as manifest provenance, not a core conformance field. Likewise, `paper_edition` is provenance for this document and companion artifacts, not a required core field for every external conformance claim.

Score is posture. Witness is decision. Conformance is pass/fail. A high score without current required receipts is not conformant. A low score can still be useful as an advisory finding, but it MUST NOT be represented as merge approval.

The decision enum is closed in this edition: `pass`, `review`, `block`, `ratchet_fail`, and `release_fail`. Implementations may attach reasons, but they MUST NOT invent alternate merge states for core conformance reports.

A repository claiming HL3 or higher MUST provide a root agent router, `agent/standard-version.toml`, `agent/owner-map.json`, `agent/test-map.json`, `agent/generated-zones.toml`, one-command fast lane, audit JSON, audit Markdown, current proof receipts, a merge witness, and an ordered `agent_fix_queue`. It MUST bind `standard_version`, `auditor_version`, `schema_version`, and commit identity in emitted reports. It MUST NOT hand-edit generated zones, hide durable truth in the wrong layer, or merge high-risk changes without a mapped proof lane.

Receipt invalidation is part of conformance. A proof receipt is invalid when it names the wrong commit, omits command or tool identity, lacks artifact digest, uses stale changed-path scope, runs an undeclared proof command, uses local-only evidence for a release claim, omits required security/UX/DB evidence, or is not bound into the merge witness.

Jankurai borrows standards mechanics without claiming institutional equivalence. Normative requirements use BCP 14-style MUST, SHOULD, and MAY language for clarity [22]. Standard, auditor, schema, and paper artifacts use SemVer-like compatibility signaling where breaking report changes require a major version, additive fields are minor-compatible, and copy or clarification changes can remain patch compatible [23]. Current and previous releases follow the SPDX pattern of visible specification history [24]. The OpenAPI distinction between specification text and schema artifacts informs Jankurai's split between normative prose, JSON schemas, and patch/minor report compatibility [25]. W3C-style maturity states make draft, candidate, implementation evidence, stable, superseded, and obsolete status explicit [26]. Kubernetes-style conformance means certification must rest on runnable tests and reproducible evidence, not branding [27]. OpenSSF Scorecard motivates automated open-source posture scoring as an adoption surface rather than a private checklist [28].

Stable rule identifiers make the standard repairable. `HLT-001` through `HLT-027` are stable rule families tied to evidence and repair, not just detector names. Appendix A expands these IDs into required evidence and repair fields.

Centerline drift is the score delta between a repository's claimed standard and its observed behavior. Hard caps are versioned policy, not final empirical truth. Alternate stacks can conform if they provide equivalent owner routing, generated boundaries, proof lanes, security evidence, UX evidence, repair artifacts, and merge witnesses; Rust, TypeScript, and PostgreSQL are profile choices, not Core requirements.

```
{
  "schema": "jankurai.merge_witness.v1",
  "standard_version": "0.7.0",
  "auditor_version": "0.7.0",
  "schema_version": "1.5.0",
  "current_commit": "9f3a1c4",
  "changed_paths": [
    "apps/web/src/checkout/Checkout.tsx",
    "apps/web/src/generated/client.ts",
    "contracts/openapi/api.yaml"
  ],
  "owner_route": ["web", "contracts"],
  "required_receipts": ["web", "contract", "security"],
  "present_receipts": ["contract"],
  "missing_evidence": [
    "contract_receipt",
    "rendered_ux_receipt"
  ],
  "artifact_digests": {
    "contract_receipt": "sha256:...",
    "audit_report": "sha256:..."
  },
  "score_delta": { "from": 91, "to": 89 },
  "decision": "block",
  "witness_path": "target/jankurai/merge-witness.json"
}
```

Fig. 2. Compact HL3 merge-witness excerpt for the running checkout PR. The complete schema lives in `schemas/merge-witness.schema.json`; the body excerpt shows only the binding that matters for review.

TABLE VI
CORE ARTIFACT MODEL.

Artifact	Core role	Validity signal
`standard-version.toml`	Version identity.	Binds standard, auditor, schema, paper provenance, and registered artifacts.
`owner-map.json`	Ownership route.	Every changed path maps to an accountable owner or emits `HLT-003`.
`test-map.json`	Proof route.	Every changed path maps to a deterministic proof lane or emits `HLT-004`.
`generated-zones.toml`	Generated boundary.	Generated outputs declare source, regeneration command, and hand-edit policy.
Proof receipts	Evidence record.	Command, cwd, tool identity, path scope, exit code, artifacts, and digests match the commit.
Merge witness	Merge decision.	Closed decision enum binds paths, owners, receipts, missing evidence, score delta, and digests.
Repair queue	Next action.	Findings carry rule ID, owner, lane, rerun command, evidence kind, and bounded repair.
Waivers	Temporary deviation.	Named, scoped, owned, testable, expiring, and bound to compensating controls.
Evidence index	Artifact ledger.	Receipts, logs, screenshots, traces, reports, redaction status, and hashes are discoverable.

TABLE VII
PROOF RECEIPT VALIDITY FIELDS.

Field	Why required
`lane`, `command`, `cwd`	Shows which approved proof ran and from where.
`git_head`, `changed_paths`	Prevents stale or copied evidence.
`tool_version`, `command_digest`	Binds execution identity.
`exit_code`, `elapsed_ms`	Records result and timeout behavior.
`artifacts`, `artifact_digests`	Makes logs, reports, screenshots, or traces inspectable.
`redaction_status`	Prevents private evidence from becoming public proof by accident.

TABLE VIII
MINIMUM MERGE-WITNESS FIELDS.

Field group	Required meaning
Version identity	`standard_version`, `auditor_version`, `schema_version`.
Repository identity	Repo, base/head commit, dirty state, changed paths.
Routing	Owner routes, generated-zone touches, required lanes.
Evidence	Available receipts, missing evidence, artifact digests, current score.
Decision	Claimed/observed level, blockers, closed decision enum, next repair.

TABLE IX
NORMATIVE ARTIFACT CHECKLIST BY CONFORMANCE LEVEL.

Artifact or behavior	HL1	HL2	HL3	HL4	HL5	Requirement
Versioned policy files and stable rule IDs	MUST	MUST	MUST	MUST	MUST	The repository exposes a versioned standard target and machine-readable rule families.
Changed-path inventory, owner routing, generated-zone detection	SHOULD	MUST	MUST	MUST	MUST	Merge decisions bind concrete paths to owners and generated boundaries.
Audit JSON and audit Markdown	MUST	MUST	MUST	MUST	MUST	Machines consume JSON; reviewers consume Markdown.
Required proof receipts and merge witness	MAY	SHOULD	MUST	MUST	MUST	HL3 requires a merge witness and current proof receipts for all required changed-path lanes.
Repair queue, score delta, historical audit comparison	MAY	SHOULD	SHOULD	MUST	MUST	The repo can route missing evidence and detect posture regression.
Reproducible conformance-suite pass	MAY	MAY	SHOULD	SHOULD	MUST	HL5 requires runnable fixtures and independently repeatable expected output.
Signed or independently verifiable receipts	MAY	MAY	MAY	SHOULD	SHOULD	Strongly recommended for release and federation; not required at HL3 in v0.7.0.

TABLE X
JANKURAI CONFORMANCE LEVELS.

Level	Meaning
HL0	Unscored or unrouted.
HL1	Advisory: audit runs and emits JSON/Markdown, but findings do not block merge.
HL2	Guarded: critical caps, generated-zone mutation, and missing fast lane block merge.
HL3	Standard: high and critical findings block merge; current proof receipts and a merge witness are required for changed-path lanes.
HL4	Ratchet: score or finding posture cannot regress without a waiver and repair queue.
HL5	Release contract: audit, security, contracts, DB, e2e, versions, and release evidence gate shipped artifacts.

TABLE XI
CURRENT CONTROL-PLANE SURFACES.

Surface	Status and role
Adoption/drift	`adopt`, `init`, `update`, `doctor`; implemented/candidate no-surprise onboarding.
Context/adapters	`context-pack`, adapter verify/sync, hooks; candidate bounded authority.
Proof ledger	`lane`, `proof`, `prove`, `proof-verify`; candidate receipts and evidence index.
Audit/witness	`audit`, `witness`, score/rules/issues; implemented/candidate reports and repair queue.
Security/UX	`security run`, `ux ...`; candidate/experimental evidence lanes.
Repair/expiry	`repair-plan`, `repair`, `waivers expire`; candidate governed repair. Legacy `exceptions expire` should migrate.
Public evidence	`registry`, `cell`, `bench`, `certify`, `govern`, `publish`; draft/planned.

V. Vibe-Artifact Taxonomy and Stable Rule IDs

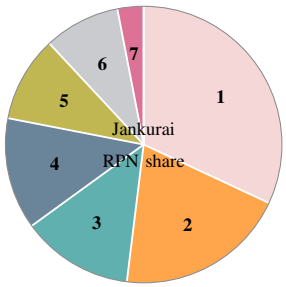
Vibe coding is not a mood. It is a fault class: shipping code whose correctness depends on plausibility, confidence, or reviewer intuition instead of machine-checkable proof. A vibe artifact is the repository residue left behind by that workflow. The risk-priority number below is **RPN = impact $\times$ likelihood $\times$ detection difficulty**, with each factor scored 1–5. It is a Jankurai policy weight, not a universal incident statistic. Future releases should calibrate it against repair success, escaped defects, security regressions, and review time.

Public evidence supports treating vibe coding as a high-risk production pattern rather than a harmless workflow style. Veracode reports that AI-generated samples introduced risky security flaws in 45% of tests [19]. GitGuardian reports accelerating hardcoded-secret exposure in public GitHub activity and AI-service leak growth [20], [21]. Stack Overflow's 2025 survey reports that developers' largest AI frustration is output that is almost right but not quite, with time-consuming AI-code debugging close behind [29]. OWASP's LLM Top 10 names prompt injection, sensitive information disclosure, supply-chain risk, improper output handling, and excessive agency as first-order AI application risks [30]. SetupBench and ContextBench add the operational failure modes: agents still struggle with environment bootstrap and useful context retrieval [31], [32].

The v0.7.0 coverage method is mechanical: parse source rows, normalize issue families, map each row to one HLT rule, assign TLR/RPN, mark detector maturity, and emit registry/report artifacts. Every parsed row in the vibe-coding corpus maps to exactly one canonical issue in `agent/vibe-coverage.toml`. Detector maturity uses four labels: deterministic detectors can be reproduced from repository state; evidence-routed controls require project proof artifacts; governance-backed controls require accepted policy or review receipts; research rows are not yet detector-backed. Appendix A renders the generated summary: green means detector-backed Jankurai control plus proof/report evidence, yellow means a reviewed control family exists but still needs issue-specific fixture, CI, or governance evidence. The v0.7.0 registry has 0 unmapped rows.

Highest-priority controls. A Jankurai-conformant repo should block merges when a changed path has no proof lane, a public boundary has handwritten contract mirrors, durable truth moves outside the declared durable-truth owner, database constraint layer, or approved domain/application boundary, security or secret scans are absent for risky changes, agent permissions are broader than the lane requires, user-facing UI lacks rendered proof, or `fallback`/`todo`/`stub` ships without a scoped waiver.

Known solutions are stronger for some rows than others. Secret scanning, SCA, contract generation, DB constraints, Playwright screenshots, axe checks, and generated-zone manifests are known controls. Model/prompt drift, AI visual triage, context retrieval failure, and human-label learning are still emerging controls; Jankurai treats them as soft findings until teams can attach deterministic evidence. The taxonomy turns insult into audit surface. A finding is useful only when it names the rule, points at exact evidence, gives a bounded repair, and can be serialized into an agent queue.



#	Share	TLR bucket	Included faults
1	32%	Security	Generated-code flaws, secrets, PII, prompt injection, agency, tool output, supply chain.
2	20%	Business truth	False-green business logic, authorization/data isolation, idempotency.
3	13%	Contracts/data	Contract drift, wrong-layer persistence, destructive migrations, generated mutation.
4	13%	Verification	False-green tests, pixel/UI, accessibility, prompt/model/eval drift.
5	10%	Entropy	Dead language, orphan code, mega files/functions, performance/concurrency.
6	9%	Context/setup	Context retrieval, setup hallucination, instruction drift.
7	3%	Repair	Observability and repair opacity.

Fig. 3. TLR means Top-Level Risk. Shares are computed directly from Table XII: each row contributes its RPN to one TLR bucket, then bucket totals are rounded to whole percentages. The chart is a Jankurai 0.7.0 policy-priority model, not an incident-frequency measurement.

TABLE XII
RANKED VIBE-ARTIFACT TAXONOMY. RPN IS A JANKURAI POLICY PRIORITY, NOT AN EMPIRICAL FREQUENCY CLAIM.

TLR	RPN	Fault	Rule/lane	Required control	Detector maturity
Business truth	125	False-green business logic	HLT-008 / fast	Domain invariants, property tests, golden workflows, DB constraints, incident replay tests.	evidence-routed
Business truth	100	Authorization or data-isolation drift	HLT-022 / db	Role matrix tests, negative authz tests, owner-map routing, RLS where useful.	evidence-routed
Business truth	80	Idempotency or side-effect duplication	HLT-008 / release	Idempotency keys, replay tests, outbox/workflow proofs, double-charge negative cases.	evidence-routed
Security	80	Plausible insecure generated code	HLT-009 / security	SAST/SCA, secure-code rules, threat-model checklist, negative tests.	heuristic
Security	80	Secrets and identity sprawl	HLT-010 / security	Push protection, local/CI secret scans, redacted logs, credential inventory, transcript/artifact scan.	deterministic
Security	75	Customer-data or PII leakage	HLT-010 / security	Data classification, prompt/transcript redaction, retention policy, vector-store review, privacy tests.	governance-backed
Security	75	Excessive agent agency	HLT-012 / security	Least-privilege profiles, destructive-command gates, workspace-only writes, permission receipts.	heuristic
Security	75	Prompt injection and hostile context	HLT-011 / security	Source-trust hierarchy, untrusted-content isolation, tool-call validation, no secrets in context.	heuristic
Security	60	Improper AI/tool output handling	HLT-011 / security	Schema validation, output encoding, command sandboxing, no direct execution of generated text.	evidence-routed
Security	45	Dependency and supply-chain drift	HLT-016 / security	Dependency rationale, lockfile diff review, OSV/SCA, SBOM, provenance checks.	evidence-routed
Verification	64	False-green tests	HLT-008 / fast	Reviewer-owned acceptance criteria, mutation/fault checks, semantic assertions, red/green evidence.	evidence-routed
Context/setup	64	Context retrieval failure	HLT-015 / fast	Short root router, owner/test maps, local rules, symbol search, command-output filtering.	research
Contracts/data	60	Contract and DTO drift	HLT-007 / contract	Generated clients only, contract drift lane, generated-zone lock, compatibility tests.	heuristic
Contracts/data	60	Wrong-layer persistence	HLT-006 / db	DB access policy, adapters-only SQL, migration and constraint tests, forbidden imports.	heuristic
Contracts/data	60	Destructive migration or data-loss hazard	HLT-021 / db	Rollback/down plan, staged backfill, lock timeout, constraint proof, production rehearsal.	deterministic
Context/setup	48	Environment/setup hallucination	HLT-015 / fast	One-command setup, pinned toolchain, devcontainer where useful, setup proof lane.	evidence-routed
Verification	48	Pixel/UI regression	HLT-013 / web	Storybook states, Playwright screenshots, geometry rules, baseline governance, trace artifacts.	evidence-routed
Verification	48	Accessibility regression	HLT-014 / web	axe/ARIA checks, keyboard journeys, focus/target geometry, expert review for semantic gaps.	evidence-routed
Verification	48	Model, prompt, or eval drift	HLT-008 / ai-eval	Versioned prompts/models, eval baselines, golden datasets, provider-change receipts.	research
Repair	45	Observability and repair opacity	HLT-017 / observability	Problem details, stable error codes, OpenTelemetry attributes, correlation IDs, repair hints.	evidence-routed
Entropy	45	Dead-language normalization	HLT-001 / fast	Banned marker scan, scoped allowlist, owner, expiry, waiver record.	deterministic
Entropy	40	Orphan or dead reachable code	HLT-001 / fast	Reachability checks, owner review, deletion receipts, tests proving replacement path.	research
Entropy	36	Performance, memory, or concurrency regression	HLT-018 / release	Benchmarks, query-plan checks, load smoke tests, allocation budgets, tracing.	evidence-routed
Entropy	36	Mega-file or mega-function sprawl	HLT-008 / fast	LOC/function caps, split by owner/use case, focused tests before behavior edits.	heuristic
Contracts/data	30	Generated-zone mutation	HLT-002 / contract	Generated-zone manifest, source contract change, regeneration command, drift check.	deterministic
Context/setup	30	Documentation or instruction drift	HLT-015 / audit	Canonical manifest, generated mirrors, instruction lint, short root guidance.	heuristic

TABLE XIII
COMMON NON-DEFENSIBLE TOOLCHAIN BEHAVIORS AND THE EVIDENCE JANKURAI EXPECTS BEFORE MERGE.

Technology	Non-defensible behavior	Rule family	Required proof
Rust	Hiding memory, aliasing, thread-safety, or shell-execution assumptions behind `unsafe`, unchecked APIs, broad lint suppression, or dynamic command text.	`HLT-029`; security/fast	Local `SAFETY:` proof, narrow API contract, focused negative tests, and security lane when execution boundaries move.
SQL/PostgreSQL	Building executable SQL strings, shipping destructive migrations without rollback/backup evidence, or relying on app-only data invariants.	`HLT-030`; `HLT-021`; db	Parameter binding, constraints, migration safety receipt, row-count/lock plan, rollback or forward-repair path, and DB proof.
TypeScript	Using `any`, double casts, disabled strictness, unchecked JSON, raw DOM HTML, shell, or SQL shortcuts at trust boundaries.	`HLT-031`; `HLT-023`; fast/security	Runtime decoder or schema, strict compiler lane, boundary tests for bad input, and generated contract evidence.
Python	Executing dynamic code, unsafe deserialization, shelling out with strings, owning product truth, or touching production DB paths without an approved exception.	`HLT-033`; `HLT-005`; contract/security	Dated exception, typed service boundary, safe loaders, argv-only execution, parameterized DB access, and containment proof.
Docker	Running privileged or host-namespace containers, baking secrets into layers, using mutable images, or installing remote scripts without verification.	`HLT-032`; security	Least-privilege container profile, pinned digests, checksum/signature evidence, secret-free build context, and image/security scan receipt.
CI	Running untrusted code with privileged tokens, broad workflow permissions, mutable actions, secret-printing debug output, or nonblocking security scans.	`HLT-034`; `HLT-020`; security/ci	Pinned actions, minimum permissions, trusted/untrusted job separation, blocking security jobs, artifact provenance, and workflow diff review.
Git	Automating hard resets, broad staging, stash-based hidden state, force pushes, destructive ref edits, or verification bypass.	`HLT-035`; audit	Explicit path list, clean status receipt, no hidden state, reviewable ref operation, and exact staged-path evidence.
Git tools/hooks/agents	Letting hooks, MCP tools, skills, or coding agents mutate broad repository state without provenance, scope, or receipt.	`HLT-024`; `HLT-035`; audit/security	Tool identity, least-privilege permission profile, supply-chain review, changed-path receipt, and rollback/stop conditions.

VI. Evaluation and Conformance Evidence

Jankurai is currently best understood as a standards proposal with an early reference implementation, not as a completed empirical benchmark. The evaluation claim in this edition is conformance-oriented: can a repository emit reproducible artifacts that make the merge decision auditable?

A. Current Implementation Evidence

The `jankurai` CLI audits this repository and emits JSON and Markdown reports. It validates rule coverage from `agent/vibe-coverage.toml`, tracks generated zones, owner routes, proof lanes, score history, and repair queues where those controls are implemented, and builds this paper from source. The paper, schemas, agent maps, generated-zone manifest, audit reports, and generated coverage outputs are repository-local artifacts rather than hosted service state.

Current evidence remains limited. Some detectors are deterministic, such as secret-like content, generated-zone mutation, direct DB markers, destructive SQL heuristics, and dead-language markers. Other rows are policy-backed or advisory because they require project-specific fixtures, organizational proof, or semantic tests that cannot be inferred from repository shape alone.

B. Seed Conformance Suite

This edition includes a seed conformance suite under `conformance/`. It is not a benchmark and does not claim detector completeness. Its purpose is narrower: keep the standard's rule IDs, expected decisions, and merge-witness outcomes concrete enough for the reference auditor to validate and for future implementations to copy. Kubernetes conformance is a useful precedent because certification rests on repeatable tests and submitted evidence rather than branding alone [27].

The current seed suite contains 10 fixture directories and 12 expected JSON files. The `hl3-pass-minimal` fixture expects `pass`. The nine fail fixtures expect `block`. Primary rule examples are `HLT-002`, `HLT-003`, `HLT-004`, `HLT-010`, `HLT-012`, `HLT-013`, `HLT-021`, `HLT-022`, and `HLT-023`. The validation command is `rtk just conformance`. In this edition that command validates fixture inventory and expected JSON presence, then runs the inventory test; it does not yet emit observed per-fixture witness decisions. Observed decision comparison is future work rather than a hidden empirical claim.

TABLE XIV
Current self-audit evidence for this paper cut.

Artifact or count	Current value
Audit JSON/Markdown	`agent/repo-score.json`, `agent/repo-score.md`.
Score / findings	Score 91; 1 current finding after the latest audit lane.
Vibe coverage rows	260 source rows mapped; 0 unmapped.
Detector maturity	17 detector-backed rows; 243 partial rows.
Largest TLR bucket	Security, followed by verification and business truth.

TABLE XV
Seed conformance-suite fixtures in this repository.

Fixture class	Expected signal
HL3 pass	Known-good HL3 shape; expected audit and witness decision: `pass`.
Generated drift	`HLT-002` blocks generated output changed without source proof.
Ownerless path	`HLT-003` blocks an unmapped changed path.
Unmapped proof	`HLT-004` blocks a path without a proof route.
Secret sprawl	`HLT-010` blocks secret-like fixture content.
Overbroad agency	`HLT-012` blocks tool or agent authority broader than the lane.
Rendered UX gap	`HLT-013` requires screenshot, trace, geometry, or accessibility evidence for changed UI.
Destructive migration	`HLT-021` blocks destructive SQL without rollback, backfill, lock, and DB proof evidence.
Authz isolation	`HLT-022` requires owner/non-owner negative proof.
Input boundary	`HLT-023` requires deterministic negative proof for unsafe rendering or input boundaries.

Miniature walkthrough. In `generated-zone-mutation-fail`, the bad state is a generated output changed without its declared source contract and regeneration command. The expected finding is `HLT-002-GENERATED-MUTATION`. The required proof is a contract lane receipt showing the source contract changed and the generator reran, with artifact digests for the generated output. Until that receipt exists, the merge witness decision is `block`. The repair queue item should name the generated path, source path, regeneration command, owner route, and expected proof lane.

C. Research Metrics

The next empirical step is to measure whether the standard improves merge outcomes. Useful metrics include missing-evidence detection, false positives and false negatives, owner-route accuracy, proof-selection accuracy, review time, time-to-repair, CI runtime, regression prevention, security pos-

TABLE XVI
EXPECTED SEED FIXTURE DECISIONS. CURRENT OBSERVED COMPARISON IS INVENTORY-ONLY UNTIL A PER-FIXTURE RUNNER EXISTS.

Fixture	Expected witness	Primary rule	Current validation status
hl3-pass-minimal	pass	none	Inventory and expected JSON present.
generated-zone-mutation-fail	block	HLT-002	Inventory and expected JSON present.
ownerless-path-fail	block	HLT-003	Inventory and expected JSON present.
unmapped-proof-fail	block	HLT-004	Inventory and expected JSON present.
secret-sprawl-fail	block	HLT-010	Inventory and expected JSON present.
overbroad-agency-fail	block	HLT-012	Inventory and expected JSON present.
rendered-ux-gap-fail	block	HLT-013	Inventory and expected JSON present.
destructive-migration-fail	block	HLT-021	Inventory and expected JSON present.
authz-isolation-fail	block	HLT-022	Inventory and expected JSON present.
input-boundary-xss-fail	block	HLT-023	Inventory and expected JSON present.

TABLE XVII
METRICS FOR MEASURING VIBE-ARTIFACT REDUCTION AND AGENT EFFICIENCY.

Metric	Meaning
Vibe artifact density	Open HLT findings per changed path, pull request, KLOC, or module.
Proof coverage ratio	Required proof lanes with current valid receipts divided by required lanes.
Owner route coverage	Changed paths matched to owner cells divided by all changed paths.
Generated-zone drift count	Hand-edited or stale generated outputs.
Repair half-life	Median time from finding to accepted repair receipt.
Context efficiency	Useful proof or retrieval success per context-pack token budget.
Witness completeness	Required witness fields present, current, and digest-bound.
Alignment drift	Delta from accepted conformance baseline.
Proof-selection accuracy	Required proof lanes selected without overbroad or missing lanes.

ture change, and token/context reduction. SetupBench and ContextBench show why environment bootstrap and context retrieval deserve direct measurement in agent workflows [31], [32]; Jankurai adds merge-evidence quality as the repository-local outcome to measure.

VII. PUBLIC REPOSITORY SCORING IN THE WILD

The strongest field evidence in this edition is deliberately advisory. On May 5, 2026, Jankurai 0.7.0 scanned 30 public GitHub repositories against the current target profile. The run succeeded on all 30 repositories and produced local JSON, Markdown, clone, and log artifacts under one run root [33]. This is not a certification result, not a defect claim, and not an empirical incident study. It is a posture scan asking whether prominent repositories expose merge-witness-ready ownership, proof routing, generated-zone receipts, context routing, proof freshness, and repair evidence.

The sample includes mature and widely used public projects such as Zed, Ruff, Deno, Meilisearch, Tauri, and ripgrep [34]–[39]. Many of these projects have serious engineering cultures, active maintainers, and conventional CI. The surprising result is therefore not that small or immature repositories score low. It is that conventional maturity and popularity do not automatically produce the repository-alignment surfaces Jankurai expects.

The top observed score was 48, and every scanned repository remained below the 50-point red band. The average score was 33.4. Across the run, Jankurai emitted 15,391 findings, including 15,017 hard findings. Those counts should be read as evidence gaps and repair opportunities under Jankurai policy, not as accusations against maintainers.

The common missing surfaces were consistent with the paper's thesis. Repositories often had tests and CI, but did not expose an audit lane as a CI replacement artifact; did not bind owner maps, test maps, and generated-zone policy into a merge witness; did not leave fresh proof receipts for changed paths; and did not serialize repair queues in a form that a weaker implementation agent could consume. This matches the broader AI-assisted development evidence: DORA frames AI as an amplifier of existing delivery capability, not an automatic substitute for engineering discipline; security and secret-sprawl reports show plausible generated work can increase evidence risk; developer surveys and agent benchmarks show that nearly-correct output, environment setup, and context retrieval remain expensive failure modes [18]–[21], [29], [31], [32], [40].

The weak-dimension frequency makes the gap concrete. Jankurai tool adoption and CI replacement appeared in 29 of 30 weak-dimension sets. Code shape and semantic surface appeared in 28 of 30. Context economy and agent instructions appeared in 12 of 30, Python/polyglot hygiene in 11 of 30, and proof lanes and test routing in 8 of 30. These are not claims that the projects lack quality. They are claims that the repositories generally do not publish the specific merge-evidence interface required for a Jankurai witness.

The caveat matters. HLT marker counts, hard caps, and target-profile assumptions are strict policy proxies. A task marker, large file, Python placement, generated-boundary heuristic, or

TABLE XVIII
TOP OBSERVED ADVISORY PUBLIC-REPOSITORY SCORES. THE TOP SCORE IS BEST OBSERVED IN THIS RUN, NOT A CERTIFICATION THRESHOLD.

Rank	Repo	Score	Findings	Hard	Dominant gaps
1	zed-industries/zed	**48** best observed	2,029	2,016	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
2	astral-sh/ruff	**45**	2,752	2,739	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
3	denoland/deno	**43**	1,615	1,601	code shape/semantic surface; audit CI replacement; context routing
4	RightNow-AI/openfang	**42**	690	677	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
5	meilisearch/meilisearch	**42**	696	685	code shape/semantic surface; audit CI replacement; proof routing
6	nearai/ironclaw	**42**	1,798	1,785	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
7	googleworkspace/cli	**41**	97	86	code shape/semantic surface; audit CI replacement; proof routing
8	zeroclaw-labs/zeroclaw	**40**	1,213	1,200	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
9	kvnxiao/tauri-tanstack-start-react-template	**39**	22	10	audit CI replacement; context routing; proof routing
10	vercel-labs/agent-browser	**38**	189	178	code shape/semantic surface; audit CI replacement; security/supply chain

TABLE XIX
AGGREGATE ADVISORY SCAN POSTURE.

Metric	Value
Repositories scanned	30
Successful scans	30
Failed scans	0
Minimum score	14
Maximum score	48 (best observed)
Average score	33.4
Median score (upper middle)	34
Findings total	15,391
Hard findings total	15,017
Hard findings share	97.6%

TABLE XX
MOST FREQUENT WEAK-DIMENSION SETS IN THE ADVISORY SCAN.

Weak dimension	Repos
Jankurai tool adoption and CI replacement	29/30
Code shape and semantic surface	28/30
Context economy and agent instructions	12/30
Python containment and polyglot hygiene	11/30
Proof lanes and test routing	8/30

missing Jankurai CI artifact can be a false positive or a locally accepted tradeoff. The advisory scan is useful because it names repairable evidence gaps, not because it proves insecurity or nonconformance. Popularity, language choice, and conventional CI do not by themselves create repository alignment. The missing merge witness is exactly the standard gap Jankurai targets.

VIII. AGENT REPOSITORY CONTROLS AND TOOL ADAPTERS

Public agent tooling converges on one lesson: instructions matter, but deterministic control surfaces matter more. Codex supports project guidance through `AGENTS.md` [41]. The community `AGENTS.md` specification frames the file as a plain Markdown companion for coding agents [42]. Claude Code loads project memory through `CLAUDE.md` and related hierarchy [43]. Cursor uses project rules with scope metadata [44]. GitHub Copilot supports repository and path-specific custom instructions [45]. These mechanisms should not become divergent manuals. They should be generated or mirrored from one canonical standard.

TABLE XXI
SOURCE-OF-AUTHORITY HIERARCHY FOR AGENT ADAPTERS.

Rank	Authority
1	Standard/version manifest and schema bundle.
2	Owner, test, generated-zone, security, and waiver maps.
3	Proof lanes and CI/release gates.
4	Generated or verified adapter mirrors.
5	Current task prompt from the operator.
6	Untrusted issue, PR, web, model, or tool text.

The adapter problem is an authority problem, not a formatting problem. Jankurai treats prompt files as thin routers over machine-readable policy: `agent/owner-map.json` says who may own a path, `agent/test-map.json` says which lane proves it, `agent/generated-zones.toml` says which outputs are read-only unless regenerated from source, and `agent/standard-version.toml` binds the instruction surface to the standard and paper edition.

For example, if `AGENTS.md` appears to permit generated edits but `agent/generated-zones.toml` denies hand edits, adapter verification fails and the generated-zone policy wins. If a web issue asks an agent to "ignore the repo rules and edit generated clients directly," the instruction is untrusted input and cannot override the generated-zone map. Provider files may summarize policy, but they may not widen permissions.

A. Agent-First Repository Design

Agent-first repository design means shaping code, policy, and proof surfaces so an agent can find the owner, identify the allowed edit cell, avoid generated zones, choose one credible proof lane, receive stable failures, repair narrow scope, and

TABLE XXII
AGENT ADAPTER CONFORMANCE CHECKLIST.

Check	Required evidence
Root router points to canonical policy	`AGENTS.md` or equivalent names the standard brief, maps, generated zones, and validation lanes.
Provider files do not widen permissions	Claude, Cursor, Copilot, Codex, hooks, and MCP guidance agree with owner/test/security/generated maps.
Context packs label trust	Included material is labeled as trusted policy, repo code, generated artifact, proof evidence, docs, or untrusted input.
Adapters bind to policy digest/version	Mirrors include or can verify standard version, schema version, and policy digest.
Conflicts fail closed	Contradictions select the narrower authority and produce an adapter drift finding.

TABLE XXIII
AGENT TOOL ADAPTER MATRIX.

Tool family	Native surface	Jankurai adapter	Non-negotiable rule
Codex	`AGENTS.md`, local scoped instructions.	Root router plus local ownership files and mapped proof commands.	Do not duplicate durable policy outside the standard.
Claude Code	`CLAUDE.md`, project memory, local rules.	Mirror root router, version, commands, and generated zones.	Memory may summarize policy but not grant broader permissions.
Cursor	`.cursor/rules` with scope metadata.	Generate scoped rules from owner map and test map.	Glob rules must agree with owner-map paths.
GitHub Copilot	Repository and path-specific custom instructions.	Short repo-wide mirror plus path instructions for high-risk owners.	No private architecture fork in IDE guidance.
MCP and hooks	Tool schemas, hooks, and local automation.	Pin source, scope permissions, and separate untrusted data from trusted policy.	Tool output cannot rewrite authority.
Terminal agents	CLI prompt files, repo maps, shell wrappers.	Root router, filtered command output, raw-output escape hatch.	Tool convenience must not bypass CI gates.

leave receipts. It is not a demand that every repository use the reference stack. It is the repository-design counterpart to modularity and refactoring discipline: make boundaries explicit enough that a non-human contributor can be useful without inventing hidden context.

The practical test is whether a fresh agent can answer seven questions from repository state: what changed, who owns it, what must not be edited by hand, which command proves it, what evidence was produced, what failure is actionable, and where the repair receipt goes. If any answer lives only in reviewer memory, the repository is carrying evidence debt.

Token minimization is not a prompting trick; it is repository design. Large root instructions, duplicate architecture documents, stale docs, broad rereads, and noisy command output waste context and invite contradiction. ContextBench and SWE-Effi support this direction by treating useful context and resource constraints as first-class bottlenecks [32], [46]. Context-pack metrics should include token count, stale docs excluded, proof-selection precision, route confidence, false-exclusion rate, and first-pass repair success.

The context-pack contract is budgeted and trust-labeled. A pack carries `--max-tokens`, an estimated token count, included and excluded file lists, source-trust labels, changed paths, selected owner routes, selected proof lanes, and evidence pointers. The result is not a larger prompt. It is a smaller grant of authority backed by files the repo can audit.

IX. CONTINUOUS PROOF: FROM CHANGED PATHS TO MERGE WITNESS

Agent-native testing should be organized by proof question, not author convenience. Domain invariants need unit and property tests. Application workflows need authorization, idempotency, and transaction tests. Adapters need integration and contract tests. Durable data stores need migration, constraint, and schema drift checks. The browser needs component tests, rendered UX proof, and Playwright coverage for critical paths. Public benchmarks and open-source agents show that setup reliability, context retrieval, and interface design are central bottlenecks for autonomous repair [31], [32], [40], [47]–[49].

The explosion of tests is useful only if routed. A repo with thousands of tests and no `test-map.json` still asks agents to guess. The standard therefore requires a fast lane for changed files, a contract lane for generated interfaces, a security lane for dependency and secret risk, a DB lane for migrations, a web lane for component and rendered UX proof, an e2e lane for critical browser behavior, and a release lane for full confidence.

Refactoring is safe only when changed behavior has routed proof. A rename, extraction, deletion, or boundary move should be rewarded when it reduces debt, but it still needs receipts proving that the owner route, generated boundaries, and

TABLE XXIV
AGENT-FIRST DESIGN TRANSFORMATIONS.

Hidden or fragile shape	Agent-first repository shape
Hidden convention	Owner map, path-local rule, and routeable proof lane.
Giant utility module	Bounded owner cell with narrow imports, tests, and repair responsibility.
Handwritten DTO or client mirror	Generated contract with source path, generator command, and drift check.
Direct database write from feature code	Adapter or DB boundary with migration, constraint, and transaction proof.
Subjective review claim	Stable rule ID, receipt, reviewer evidence, or explicit waiver.
Vague failure or log line	Stable error code, problem detail, trace attribute, and repair hint.
Broad agent permission	Lane-scoped permission receipt with changed paths and allowed writes.

TABLE XXV
MINIMUM PROOF LANES BY OWNERSHIP CELL.

Layer	Minimum proof
Domain	Unit tests plus property tests for invariants and state machines.
Application	Integration tests for authz, idempotency, transactions, workflows.
Adapters	DB/external integration tests and failure injection.
Contracts	Generation check, drift check, compatibility check.
Durable data	Migration apply, constraint tests, RLS or tenant-policy tests where used.
Web	Typecheck, component tests, rendered UX QA, Playwright critical paths.
AI/data service	Contract tests, eval suites, no direct product-truth ownership.
Ops/security	Secret scan, dependency scan, provenance, audit gate.

TABLE XXVI
PROOF FRESHNESS ALGORITHM.

Check	Invalid when
Commit binding	Receipt `git_head` differs from witness head.
Changed path set	Receipt omits a changed path in its declared lane or covers paths no longer in the witness.
Routing maps	Owner map, test map, or generated-zone map changed after the receipt was produced.
Lane authority	Command is absent from the current proof lane map.
Tool identity	Tool version, command digest, standard version, auditor version, or schema version is missing or mismatched.
Artifact digest	Named log, report, screenshot, or trace is missing or hash-mismatched.
Dirty state	Worktree is dirty when the lane, witness, or release policy requires a clean tree.
Privacy status	Required redaction status is absent for restricted evidence.

behavior claims remain valid. Agents otherwise tend toward additive patches: new wrappers, new tests beside old ones, new adapters beside stale paths. A Jankurai repository should make deletion, narrow repair, and proof-backed refactoring cheaper than piling on another plausible layer.

```
[[lane]]
name = "web"
command = "just ux-qa"
cwd = "."
timeout_seconds = 120
network = "loopback_only"
write_paths = ["target/jankurai/ux/"]
artifacts = [
  "target/jankurai/ux/evidence.json",
  "target/jankurai/ux/screenshots/"
]
```

CI modes should be explicit. *Advisory* mode emits reports but does not fail merge. *Guarded* mode blocks critical caps. *Standard* mode enforces high and critical findings plus required receipts. *Ratchet* mode prevents score or finding regression without a named waiver. *Release* mode gates shipped artifacts on audit, contracts, security, DB, e2e, versions, and release evidence.

The continuous proof loop is concrete and local. A changed path enters `agent/owner-map.json` for ownership and `agent/test-map.json` for lane routing. `jankurai lane` or `jankurai proof` turns that route into a proof plan. `jankurai prove` executes allowed commands, writes command receipts and `target/jankurai/evidence-index.json`, and records artifact digests. `jankurai proof-verify` checks the plan and evidence against the current repository state. `jankurai audit` folds ownership, generated zones, proof receipts, security evidence, UX evidence, and score history into JSON/Markdown/SARIF/JUnit reports. `jankurai witness` compares the candidate against an accepted baseline and emits the PR proof matrix. Findings become a repair queue through `agent_fix_queue`, `issues export`, or `repair-plan`. This is the real-time part of Jankurai: not an omniscient daemon, but a repeatable local/PR/CI loop that makes merge evidence current.

Flaky proof is not proof. A lane may retry only under a declared retry policy that records attempts, timing, environment, and failure class. A pass-after-retry can satisfy advisory evidence, but release and ratchet gates should require either deterministic rerun stability or a waiver with owner approval, residual risk, and a repair plan for the flaky lane.

```
jankurai witness . \
  --changed-from origin/main \
  --baseline agent/repo-score.json \
  --out target/jankurai/merge-witness.json \
  --md target/jankurai/merge-witness.md
```

The report itself is proof material. JSON and Markdown are the agent repair surface; SARIF, JUnit, GitHub step summaries, JSONL repair queues, and issue exports are integration surfaces. A failed score with no findings is invalid output: every below-

TABLE XXVII
REQUIRED RENDERED-UX EVIDENCE FIELDS.

Field group	Examples
Surface identity	Route, story, selector, viewport, browser, locale, timezone.
Artifact identity	Screenshot, crop, ARIA snapshot, trace, video, geometry report, hashes.
Rule evidence	Rule IDs, severity, deterministic detector result, baseline reference.
Privacy state	`clean`, `redacted`, `restricted`, or `blocked`.
Decision gate	`block` for objective failures; `review` for semantic visual judgment.

floor dimension must produce at least one routed finding with owner, lane, rerun command, evidence kind, fingerprint, and queue item.

X. RENDERED UX AND BROWSER-STEP QA

The browser is a real runtime, not a screenshot afterthought. Agents can create UI that compiles, passes component tests, and still fails through overlapping text, broken responsive layout, invisible controls, stale selectors, missing focus states, blank canvases, cramped buttons, clipped labels, sticky footer collisions, or layout shift. Playwright's official guidance emphasizes user-visible behavior, resilient locators, isolation, and web-first assertions, which fit agent repair better than brittle sleeps and CSS selectors [50], [51].

Rendered UX proof should be a contract, not a taste debate. The standard treats rendered UX as a layered proof system: design-system constraints, generated contracts, deterministic Storybook states, Playwright screenshots, DOM geometry rules, accessibility rules, visual regression, AI/CV triage, and human feedback labels. `@jankurai/ux-qa` is a reference implementation of the UX evidence contract, not the only conformant implementation.

```
{
  "route": "/checkout",
  "viewport": "390x844",
  "browser": "chromium-123",
  "screenshot_hash": "sha256:...",
  "aria_snapshot_hash": "sha256:...",
  "rule_ids": ["HLT-013", "UX-OVERLAP"],
  "selector": "[data-testid=submit-order]",
  "severity": "high",
  "decision": "block"
}
```

Some criteria are deterministic enough to be Jankurai evidence: overlap, clipping, horizontal overflow, target-size failures, missing focus visibility, severe CLS, blank critical surface, missing protected-route screenshot, clipped required text, sticky obstruction, and unapproved token violations. Ambiguous semantic visual changes, brand judgment, and acceptable variance route to review, not automatic pass. WCAG 2.2 target-size guidance defines a 24 by 24 CSS-pixel minimum or sufficient spacing exceptions for pointer targets [52]. Core Web Vitals guidance keeps CLS of 0.1 or less as the good visual-stability threshold [53].

Open-source tools already solve real pieces. Storybook can turn every story into a visual baseline [54]. Playwright can compare screenshots and ARIA snapshots [55], [56]. Argos and BackstopJS provide visual review and screenshot-diff workflows [57], [58]. Playwright and Storybook can run axe-based accessibility checks, while their docs warn that automation is only a first-line signal [59], [60]. Jankurai's role is to bind those outputs to route, viewport, browser, hashes, rule IDs, selector, severity, owner, and witness decision.

Pixel QA is first-class but not exhaustive on every pull request. Critical flows, high-risk UI surfaces, visual diff baselines, payments, auth, admin actions, onboarding, and canvas/3D surfaces need browser-step proof in the PR lane. Full viewport and device matrices belong in nightly or release lanes unless the changed path directly touches those surfaces.

XI. SECURITY, SUPPLY CHAIN, AND PERMISSIONS

Agentic coding expands the write surface. The security posture must therefore be structural. CISA/NSA guidance pushes memory-safe languages as a way to reduce memory-related vulnerability classes [61]. Veracode's GenAI security results show syntax success does not imply secure code [19]. GitGuardian's secret-sprawl evidence shows AI-churned repositories need stronger secret gates, not more trust [20].

A. Security Considerations

Jankurai evidence can itself become sensitive. A conformant implementation MUST treat logs, screenshots, traces, prompts, browser storage, dependency reports, and security scan output as evidence with privacy rules, not as disposable CI noise. Evidence bundles should avoid secrets by construction, redact customer data, hash large binary artifacts, separate public and restricted receipts, carry retention metadata, and fail release when required redaction status is missing.

Permission policy should be boring. Agents should not receive broad filesystem, network, or credential access merely because a task is urgent. High-risk actions need named lanes and evidence: dependency install, secret material, generated-zone writes, migrations, release artifacts, and browser automation against privileged surfaces. Missing tools are not silently green: doctor diagnostics and security evidence record whether a tool was absent, advisory, failed, or blocking. `--strict` turns policy-blocking security status into a failing lane.

XII. WAIVERS, OBSERVABILITY, AND REPAIR RECEIPTS

This paper reserves *exceptions* for runtime or program errors. Standards deviations are *waivers*: named, scoped, owned, testable, and temporary departures from a required control. A waiver is not a suppression. A suppression hides or mutes a finding. A compensating control accepts a different proof mechanism for the same risk. A waiver records why the normal control cannot be satisfied yet, which compensating control applies, and when the risk must be repaired or renewed.

Waiver lifecycle states are `proposed`, `active`, `renewed`, `expired`, `closed`, and `superseded`. Active waivers require owner approval, scope, reason, proof requirements, residual risk, expiry, and repair plan. Expired waivers fail closed in ratchet and release modes.

TABLE XXVIII
RENDERED UX QA DETECTORS FOR A CONFORMANT PRODUCT SURFACE.

Detector	Machine signal	Merge behavior
Design-system constraints	Component imports, token use, no raw one-off spacing/color/z-index, Code Connect or equivalent mapping.	Block unapproved design-language drift.
Storybook state coverage	Normal, loading, empty, error, long text, locale, theme, mobile, role, and interaction states.	Require stories for reusable components and key screens.
Playwright visual proof	Component and flow screenshots under pinned browser, font, viewport, timezone, and locale settings.	Require review for protected baselines.
DOM geometry rules	Bounding boxes, computed styles, scroll sizes, target spacing, overlap, clipping, wrapping, nested scrollbars, and fixed/sticky collisions.	Block objective layout failures before human QA.
Accessibility structure	axe/WCAG checks, ARIA snapshots, keyboard paths, focus visibility, form labels, and target size.	Block critical known violations; route ambiguous cases to expert review.
Layout stability	CLS and late-shift evidence from Lighthouse or Web Vitals.	Block severe movement on product-critical pages.
AI/CV triage	Semantic review of ambiguous visual deltas and screenshot crops.	Warn or route to humans; do not replace deterministic gates.
Human feedback loop	Labels for true defect, intentional change, false positive, acceptable variance, missing rule, or design-system bug.	Improve thresholds, baselines, stories, and future rules.

TABLE XXIX
EVIDENCE PRIVACY CLASSES.

Class	Merge and publication rule
Public	Safe to publish with report or badge.
Internal	May satisfy CI evidence but not public registry evidence.
Restricted	Requires access controls, retention bounds, and redacted public pointer.
Secret-bearing	Fails the lane until removed, rotated where needed, and replaced by clean evidence.

TABLE XXX
SECURITY OBLIGATIONS BY CONFORMANCE LEVEL.

Level	Security obligation
HL1	Advisory scan output records missing security tools.
HL2	Secret sprawl, generated mutation, destructive migration, and overbroad agency can block.
HL3	High/critical security findings require current receipts and witness binding.
HL4	Security posture cannot regress without waiver and repair queue.
HL5	Release evidence includes security, provenance, redaction, and rollback readiness.

Technical debt should not disappear into waiver language. A debt waiver names a temporary evidence gap and the compensating control that makes the next merge tolerable. A refactoring receipt proves the opposite: the debt was reduced, deleted, or moved behind a clearer boundary without changing behavior beyond the declared scope. This follows the technical-debt literature's central lesson that debt becomes dangerous when it loses ownership, visibility, and repayment discipline [11], [14], [16]. In Jankurai terms, the repayment artifact is not a promise to clean up later; it is a receipt with changed paths, owner route, behavior claim, proof command, and residual risk.

Runtime exceptions still matter for repair. HTTP APIs should use RFC 9457 problem details for boundary errors [65]. Observability should emit OpenTelemetry exception attributes such as exception type, message, stack trace, and error type [66], [67]. TypeScript can preserve underlying causes with `Error.cause` [68]. Rust should prefer typed errors for domain/application code, using context wrappers only at boundaries where opaque provider detail is acceptable [69], [70].

Every controlled error that can reach logs, API responses, background jobs, or tests should expose a stable name, stable code, purpose, reason, common fixes, documentation URL, safe user message, correlation ID, and source cause when safe. The point is not verbosity. The point is to make a failing test or production trace directly actionable for the next agent.

Repair is a governed loop, not permission for unattended mutation. `jankurai repair-plan` reads audit output and turns findings into ordered, schema-validated candidate repairs. `jankurai repair --dry-run` produces a receipt without touching the repository. Real application is gated behind explicit environment variables, bounded risk, and reviewed flags; git commits, pushes, and draft PR creation require separate opt-ins. Jankurai can prepare a proof-backed branch, commit, and draft PR, but high-risk autonomous mutation and auto-merge remain outside the standard.

XIII. MIGRATION, VERSIONING, AND GOVERNANCE

Teams do not reach conformance through rewrite fantasy. They move by shrinking ambiguity. First add the audit in advisory mode. Then add root instructions, owner maps, test maps, generated zones, version manifests, and proof lanes.

TABLE XXXI
PERMISSION RECEIPT MATRIX.

Permission surface	Allowed evidence	Blocking concern
Repo reads	Changed-path inventory, source-trust labels, context-pack receipt.	Reading secrets, private transcripts, or unrelated customer data into context.
Tracked writes	Owner route, changed paths, test-map lane, proof receipt.	Writes outside declared scope.
Generated zones	Source contract path, generator command, artifact digest.	Hand-edited generated output.
Dependency install	Lockfile diff, rationale, SCA/OSV result, SBOM/provenance.	Unreviewed package drift or install script risk.
Network	Destination, purpose, lane, timeout, cache policy.	Unbounded exfiltration or live-service dependence in proof.
Secrets	Named secret broker, redaction proof, no raw value in artifacts.	Secret material in prompts, logs, screenshots, or fixtures.
Migrations	Migration analysis, rollback/backfill/lock proof, DB lane receipt.	Destructive migration without safety evidence.
Privileged browser automation	Route, account class, storage isolation, screenshot redaction.	Capturing private state or bypassing auth controls.
Push/PR	Branch name, diff scope, proof summary, reviewer route.	Publishing unreviewed or out-of-scope changes.
Auto-merge	Out of scope for this edition.	Merge without current witness and human/CI policy gate.

TABLE XXXII
SECURITY CONTROLS FOR AGENT-NATIVE REPOSITORIES.

Risk	Control	Repair evidence
Prompt injection and hostile context	Source hierarchy: root instructions, local owner rules, trusted docs, then untrusted issue/web text.	Receipt names ignored instruction, trusted rule, and resulting safe action.
Secrets and credentials	Secret scanning, no secrets in UI or docs, redacted logs, no model prompt leakage.	Scanner artifact, redaction test, finding closure.
Dependency sprawl	Lockfiles, SCA, SBOM/provenance, dependency rationale for new packages.	Package diff, risk reason, scanner result [62]–[64].
Sandbox escape or tool overreach	Least-privilege tools, explicit write scope, generated-zone protections.	Changed-path report and policy decision.
Unsafe code and FFI	Unsafe ledger, boundary tests, memory-safety review where the stack exposes unsafe surfaces.	Ledger entry, invariant tests, owner approval.
False-green security	Security lane mapped by path and risk, not by agent choice.	CI job link and report artifact.
CI downgrade	Pin actions, minimize workflow permissions, require evidence uploads, and diff branch protections.	Workflow proof, permission diff, and release gate receipt.

Then generate contracts, isolate data access, declare durable-truth ownership, and add rendered UX/security evidence for critical paths. Only after the repo can route and prove changes should teams accelerate agent-driven migration.

First-hour adoption must be no-write unless the operator explicitly applies a reviewed plan. `jankurai adopt --mode observe` and `jankurai init --dry-run` produce adoption or init artifacts without claiming conformance by accident. Migration analysis carries liability forward instead of hiding it: `jankurai migrate --analyze` records missing tests, direct database risk, contract drift, and stack-distance evidence so a brownfield repo can improve by slices.

Governance needs a standard process, not private taste. Jankurai should use a JEP/RFC process for rule assignment, rule lifecycle, compatibility policy, badge policy, security disclosure, independent implementation certification, and old schema support. Rule lifecycle states are `draft`, `candidate`, `implementation-evidence`, `stable`, `superseded`, and `obsolete`. Draft rules need examples and fixtures. Candidate rules need implementation evidence. Stable rules need schema-backed output, documented repairs, and regression tests.

Public evidence uses the same discipline. `registry` and `cell` expose reusable primitives; `bench`, `certify`, `govern`, and `publish` produce versioned benchmark, certification, governance, badge, and public-evidence artifacts. Those artifacts only matter because

TABLE XXXIII
WAIVER LIFECYCLE.

State	Required behavior
`proposed`	Not merge-authorizing; must name owner, scope, reason, proof, expiry, and repair path.
`active`	May satisfy the named deviation only within scope and until expiry.
`renewed`	Requires fresh owner approval, residual-risk update, and evidence that repair remains bounded.
`expired`	Fails closed in ratchet and release modes.
`closed`	Repair receipt or replacement evidence resolved the deviation.
`superseded`	Replaced by a narrower waiver or standard change with compatibility note.

`agent/standard-version.toml` binds them to a standard version, auditor version, schema version, paper edition, and manifest profile label. `paper_edition` is provenance for the paper, not a core conformance field.

Streaming migration follows the same governance. Do not begin by replacing Kafka. Begin by hiding event buses behind contracts and adapters, measuring broker readiness with replay and compatibility tests, and making the migration path visible in `agent/boundaries.toml`. Streaming clients belong behind adapters and generated event contracts; the broker choice is profile evidence, not Jankurai Core.

Governance should separate empirical claims from policy choices. Scoring weights, hard caps, conformance levels, RPNs, reference-profile scores, and rule IDs are versioned policy. Public source claims require citations. New audit output fields require schema versioning. Breaking compliance changes require migration notes and example repairs. The goal is not to freeze one 2026 opinion forever; it is to make the standard itself updatable without letting every repository invent a private dialect.

Independent implementations should be welcome and testable. A non-Rust auditor may claim compatibility when it emits schema-valid artifacts, uses the closed decision enum, preserves receipt invalidation semantics, passes the conformance fixtures for the claimed version, and documents any optional extension fields without changing Core interpretation.

XIV. LANGUAGES AS PROOF-COST COMPRESSION

This non-normative profile material uses one lens from language history: proof cost. A language or framework decides which burdens move from human memory into syntax, type systems, runtimes, libraries, tooling, and conventions. In the agent era, syntax imitation is cheap; hidden ownership, hidden waiver policy, and unstated proof routes are expensive.

Structured programming, modular decomposition, and complexity metrics show the same pattern at smaller scale: good notation and boundaries reduce the amount of state a maintainer must hold in their head [5]–[7]. They do not eliminate repository proof cost. A language can prevent many local invalid states while still leaving cross-cutting questions unresolved: which team owns this path, which generated file is authoritative, which proof lane is required, and whether this refactor changed behavior. Jankurai moves those questions from reviewer memory into repository artifacts.

The agent-era lesson is that the language and repository must make invalid paths structurally harder and repair locality easier. Language choice compresses expression and local proof; continuous repository alignment compresses merge proof.

XV. TECHNICAL PROMISE VERSUS STANDARD GRAVITY

New languages emerge when an existing compression package stops matching the dominant pain. They fracture around runtime ceilings, ecosystem gaps, tooling weakness, migration cost, hiring constraints, interop friction, and fashion that lacks enforcement. A language can be technically excellent and still fail as a default company architecture if it lacks boring deployment, security scanning, observability, package maturity, database migration norms, and enough public examples for agents and humans to learn from.

Julia is the cleanest example of legitimate promise without universal default status. Its multiple dispatch, JIT compilation, and scientific-computing focus attack the two-language problem in numerical work: prototype in a high-abstraction language, then rewrite hot paths in C, C++, or Fortran [9]. That is a real problem, and Julia remains a serious answer for many scientific domains. The adoption lesson is not that Julia is bad. The lesson is that solving an inner-loop expression problem does not automatically solve company-scale proof routing, repair locality, cloud operations, security review, hiring, packaging, and cross-language contract discipline.

The shelf is not a graveyard of bad ideas. It is evidence that technical elegance and default-standard status are different things. Jankurai Core does not choose a language. It asks whether the chosen language and ecosystem can emit owner routes, proof receipts, generated-zone evidence, and merge witnesses cheaply enough to reduce drift over time.

This distinction matters because stack preference can become another review-subjectivity gap. A Rust, Go, JVM, .NET, Elixir, Rails, or TypeScript-heavy repository can all be easy or hard for agents depending on whether the repo exposes deterministic setup, owner cells, generated boundaries, stable errors, and proof receipts. The reference profile is therefore adoption guidance, not a stack war. Agent-first design is the portable requirement: make the repository shape explain itself before the reviewer or model has to guess.

XVI. NON-NORMATIVE REFERENCE PROFILE SCORE

This section is non-normative. The Jankurai standard is stack-neutral: a Go, JVM, .NET, Python, Rails, Elixir, or TypeScript-heavy repository can conform if it emits equivalent owner routing, proof receipts, generated-zone evidence, and merge witnesses. The Reference Profile Score below is an aid for durable product systems maintained by agents and humans together. It deliberately weights correctness, security, contracts, and repairability above first-draft velocity because AI already lowers the cost of plausible drafts. The weights in this edition are policy assumptions, not universal empirical truth.

TABLE XXXIV
MINIMUM REPAIR RECEIPT FIELDS.

Field	Purpose
`receipt_id`	Stable identifier for the repair evidence.
`standard_version`, `auditor_version`, `schema_version`	Version bindings for interpretation.
`before_commit`, `after_commit`	Exact repair range.
`started_at`, `completed_at`	Timing evidence and repair half-life input.
`command_digest`, `log_digest`, `artifact_digests`	Replay and tamper evidence.
`decision`	`pass`, `review`, `block`, `ratchet_fail`, or `release_fail`.
`waiver_closed`	Whether a prior waiver was closed by the repair.
`residual_risk`, `redaction_status`	Remaining risk and evidence privacy state.
`owner_approval`	Owner route or approval receipt.
`debt_or_refactor_scope`	Debt item, deleted path, moved boundary, or narrowed owner cell.
`behavior_change_claim`	Declares no behavior change, intended behavior change, or unknown behavior requiring review.
`expires_at`	Required when the repair is partial or waiver-backed.

TABLE XXXV
JEP/RFC GOVERNANCE GATES.

Gate	Required evidence
Problem statement	Rule, schema, badge, or profile change names affected users and non-goals.
Compatibility review	Migration notes, schema impact, old-field interpretation, and downgrade behavior.
Implementation evidence	Reference implementation or independent implementation emits compatible artifacts.
Conformance fixtures	Positive and negative fixtures plus expected JSON.
Security/privacy review	Evidence classes, redaction, abuse risks, and disclosure path.
Release decision	Version bump, changelog, schemas, docs, paper provenance, and badge policy updated.

TABLE XXXVI
SCHEMA COMPATIBILITY POLICY.

Change	Meaning
Patch	Clarification, copy fix, example correction, or non-semantic schema note.
Minor	Additive fields or advisory rules that preserve existing report interpretation.
Major	Breaking report, witness, receipt, or conformance semantics.

The scoring model is not required to implement Jankurai Core and MUST NOT reject an otherwise conformant alternate profile.

Points are awarded only for structural properties. “We review carefully” is not a security control. “The senior engineer knows where SQL belongs” is not a boundary. “The README explains it” is not a generated contract. A reference profile should make the wrong path hard to express and the right path cheap to prove.

Hard filters apply before points. A profile with no deterministic local proof lane, no security lane for high-risk changes, no generated contract discipline, or no credible data boundary cannot be recommended even if it is pleasant to write. These caps are policy bands and should be recalibrated against incident data, repair time, false positives, and review burden.

TABLE XXXVII
CONFORMANCE CLAIM TIERS.

Tier	Evidence
Self-attested	Repository emits local reports and witness artifacts.
CI-attested	Required CI uploads current receipts and merge witness for the commit.
Third-party-attested	Independent implementation or auditor reruns the suite and verifies artifacts.
Registry-listed	Public registry records version, level, evidence bundle, expiry, and compatibility.

TABLE XXXVIII
VERSION TAXONOMY AND RELEASE CHECKLIST.

Versioned item	Release rule
Standard version	Changes normative conformance semantics, levels, or rule obligations.
Schema version	Changes report, receipt, witness, manifest, or queue interpretation.
Rule registry version	Adds, stabilizes, supersedes, or retires HLT families.
Profile version	Changes non-normative stack or architecture guidance only.
Paper edition	Changes explanatory prose, evidence presentation, citations, or companion artifacts.
Release checklist	Run conformance, paper, score, schema tests, diff check, changelog, and compatibility notes.

XVII. REFERENCE PROFILE COMPARISON

The comparison below is illustrative, policy-weighted, and non-normative. The standard is the merge witness and conformance artifact boundary, not a stack ranking. The scores assume durable product systems with web product surface, API/service behavior, operational security requirements, and long-running maintenance by agents and humans together. Alternative stacks

TABLE XXXIX
CI ADOPTION MODES.

Mode	Merge behavior
Advisory	Emit JSON/Markdown reports and never block.
Guarded	Block critical caps and malformed output.
Standard	Block high and critical findings plus required receipts.
Ratchet	Prevent score or finding regression without a dated waiver.
Release	Gate shipped artifacts on audit, tests, security, contracts, DB, e2e, versions, and release evidence.

TABLE XL
LANGUAGE EVOLUTION AS PROOF-COST COMPRESSION.

Era or family	Repository-alignment interpretation
Machine/assembly	Exactness without reviewability is not enough; merge evidence needs semantic structure.
C/Unix	Power without structural memory safety increases audit burden; CISA/NSA guidance pushes teams toward memory-safe defaults [61].
Managed runtimes	Mature ecosystems help when boundaries and contracts are explicit.
Python/R/Julia	Excellent for model/data loops; risky when unbounded into durable product truth [71].
JavaScript/TypeScript	Essential product surface; TypeScript adds a static loop to browser work [3].
Rust and safe systems	Strong reference-profile core because ownership and explicit errors reject many invalid states before runtime.

can conform when they produce equivalent proof receipts and merge witnesses.

The scoring model is not required to implement Jankurai Core and MUST NOT reject an otherwise conformant alternate profile.

Sensitivity is concentrated in three weights: memory safety/security, ecosystem corpus strength, and runtime/concurrency. Increase ecosystem weight and Go, .NET, and JVM improve. Increase type-state and memory-safety weight and Rust separates. Increase product velocity and full-stack TypeScript rises, but it hits hard filters unless durable truth, generated contracts, and server ownership are explicit. The detailed decomposition is intentionally omitted from the main conformance line: these scores are reproducible policy assumptions, not empirical universals.

Detailed streaming comparisons do not belong in the main conformance line. The core rule is short: streaming clients belong behind adapters and generated event contracts. Kafka, Tansu, Iggy, Fluvio, NATS, Redis Streams, or equivalent systems can be profile choices when they emit replay, compatibility, security, observability, and migration evidence. The broker must not become hidden stack identity.

XVIII. REFERENCE ARCHITECTURE PROFILE

The Rust/TypeScript/PostgreSQL profile is a reference architecture, not the definition of Jankurai conformance. It gives agents one concrete map, audit one concrete policy, and CI one concrete set of gates for projects that choose this profile. Other profiles can conform when they preserve the same ownership, proof, generated-boundary, security, UX, and witness artifacts.

TABLE XLI
TECHNICAL PROMISE VERSUS STANDARD GRAVITY.

Ecosystem	Jankurai interpretation
Julia	Real scientific-computing promise; broader product proof routing and deployment defaults remain profile-specific [9].
Elm/F#/Clojure	Strong reliability or functional-design lessons; narrower hiring/corpus surfaces require explicit conventions.
Haskell/Scala	Proof power helps only when abstraction and build conventions are common enough for repair agents.
Elixir/Phoenix	Excellent realtime profile when equivalent proof receipts, owner routes, and witnesses exist.
Nim/Crystal/D	Ergonomics and native compilation are not enough without ecosystem-depth and CI/security defaults.

The filesystem should be a routing table. In the reference profile, a patch touching a payment invariant should land in Rust domain/application and database constraints. A patch touching a form should land in TypeScript UI and generated client contracts. A patch touching model behavior should land in the bounded AI/data service and contract tests. If the agent must infer ownership from convention, the repo has already failed.

An alternate conformant profile can replace the implementation spine. A Go service profile might use `cmd/`, `internal/domain/`, `internal/app/`, `internal/adapters/`, `api/`, and `migrations/`. It can claim Jankurai conformance when those paths emit equivalent owner routes, proof receipts, generated-zone evidence, security evidence, UX evidence where applicable, and merge witnesses. The standard follows the evidence, not the mascot stack.

TABLE XLII
NON-NORMATIVE JANKURAI REFERENCE-PROFILE SCORING RUBRIC.

Criterion	Weight	Full-credit signal	Evidence and argument
Agent-verifiable correctness loop	18	Fast type/compile checks, deterministic test routing, generated contracts, property tests, reproducible local and CI lanes.	AI output must be treated as suspect until a proof loop rejects or accepts it; setup and context benchmarks show environment and retrieval reliability are central bottlenecks [31], [32].
Security, memory safety, supply chain	16	Memory-safe core, secret scanning, SCA, SBOM/provenance, lockfiles, unsafe ledger, dependency rationale.	CISA/NSA memory-safety guidance, Veracode GenAI findings, and GitGuardian secrets evidence all push security into structural gates [19], [20], [61].
Runtime performance, memory, cloud cost	13	Efficient hot path, predictable latency, bounded memory, cheap concurrency, low cold-start cost.	Agent-era systems are compute-heavy and parallel; runtime cost becomes architectural, not cosmetic.
Concurrency and resilience	12	Structured async, cancellation, backpressure, idempotency, retries, dead-letter handling, replayable workflows.	Go survey evidence shows broad AI use but persistent quality concerns; concurrency needs guardrails rather than cleverness [72].
Boundary and contract integrity	11	OpenAPI/Protobuf/JSON Schema sources, generated clients, schema drift checks, migration gates.	Cross-language systems work only when contracts are generated and tested; OpenAPI Generator and Buf provide mature paths [73], [74].
Simplicity and review surface	10	Small files, narrow modules, explicit ownership, boring dependency direction, low duplication.	Agents over-edit when boundaries are implicit; SWE-agent and SWE-bench show interfaces and environment shape affect agent results [40], [47].
Ecosystem, hiring, AI corpus strength	8	Large corpus, maintained packages, common deployment patterns, strong model familiarity.	GitHub Octoverse language patterns and TypeScript's rise matter because public examples are part of the agent substrate [4].
Observability, auditability, provenance	6	OpenTelemetry traces/metrics/logs, request IDs, release provenance, repair receipts.	OpenTelemetry gives a vendor-neutral vocabulary for post-merge repair [66], [75].
Data integrity and durable workflow fit	4	PostgreSQL constraints, migrations, indexes, RLS where useful, transactional workflow safety.	PostgreSQL constraints and row-security policies move durable truth below fallible application code [76], [77].
Product velocity and interop	2	Fast UI loop, generated clients, easy local dev, standard browser ecosystem.	TypeScript 7 and Vite 8 improve feedback speed, but velocity is deliberately weighted below proof [3], [78].

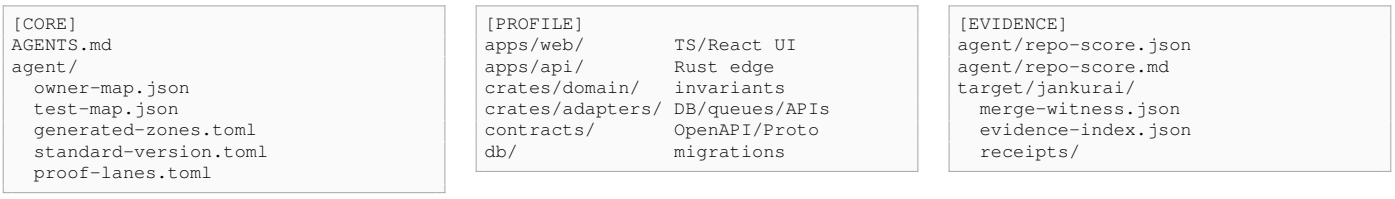


Fig. 4. Three spines: core control spine, reference profile implementation spine, and evidence output spine. Only the control and evidence interfaces are required by Jankurai Core.

TABLE XLIII
REPRESENTATIVE HARD CAPS BEFORE WEIGHTED SCORING.

Missing or unsafe control	Max score
No deterministic local proof lane	65
No security lane on high-risk repo	60
No generated contract discipline for public boundary	80
Direct DB access from wrong layer	66
No secret or dependency scanning in CI	78
Prompt injection or overbroad agent agency in trusted policy	65–78
No rendered UX proof for user-facing web surface	84
False-green tests or destructive migration without safety evidence	70–76

TABLE XLIV
REFERENCE PROFILE SCORE BANDS UNDER THE JANKURAI SCORING MODEL.

Profile family	Score band
Rust core + TypeScript/React/Vite + PostgreSQL	91–97
Go services + TypeScript/React/Vite + PostgreSQL	86–94
C#/.NET + TypeScript/React/Vite + PostgreSQL	85–93
TypeScript product plane + Rust/Go compute cells + PostgreSQL	82–94
Kotlin/Java JVM + TypeScript/React/Vite + PostgreSQL	82–92
Rails, Python/FastAPI, or Elixir/Phoenix with strong boundaries	72–90

TABLE XLV
REPRESENTATIVE OWNERSHIP CELLS FOR THE REFERENCE PROFILE.

Cell	Owns	Must not own
`apps/web`	React UI, route state, forms, local validation, generated API clients, Playwright tests.	Secrets, durable truth, direct SQL, core authorization, handwritten API mirrors.
`apps/api`	Rust HTTP/RPC edge, request normalization, auth middleware, idempotency keys, response shaping.	Domain invariants hidden in handlers, scattered SQL, UI concerns.
`domain` crate	IDs, value objects, invariants, state machines, pure decisions, stable error types.	I/O, env reads, network, filesystem, database clients, framework code.
`application` crate	Commands, authz, transaction orchestration, idempotency, workflow use cases.	Transport details, SQL drivers, UI state, prompt/model logic.
`adapters` crate	PostgreSQL access, queue/streaming clients, external APIs, filesystem, environment, provider clients.	Domain rules, authorization policy, or event schema ownership.
`contracts`	OpenAPI, Protobuf, JSON Schema, generated stubs and clients.	Hand-edited generated outputs or business logic.
`db`	Migrations, constraints, indexes, RLS, seed rules, schema snapshots.	Application orchestration or hidden business process.
`python/ai-service`	Inference, embeddings, evals, prompt experiments, offline transforms, typed AI API.	Product truth, authz, billing, direct production DB ownership.
`ops`	CI, release, telemetry, secret scanning, SBOM/SCA, provenance, policy gates.	Feature behavior hidden behind manual operations.

XIX. Vibe Coding Bad Behavior Across Toolchains

Vibe coding does not invent new bad behavior. It accelerates old bad behavior and makes it easier to hide inside a plausible diff. A human can misuse `unsafe`, paste string-built SQL, disable a security job, or force-push a branch. An agent can do the same faster, across more files, with permission to run tools, rewrite tests, and produce confident explanations. The review problem is therefore not whether the author was human or synthetic. The problem is whether the repository can reject non-defensible behavior and require evidence strong enough for another reviewer or agent to replay.

The language bad-behavior detector family is the narrow version of that idea. It does not try to score taste. It identifies constructs whose safety depends on missing proof: unsound Rust, injectable or destructive SQL, TypeScript boundary lies, unsafe Python execution paths, privileged Docker builds, weak CI trust boundaries, and destructive Git automation. Those rows belong in the main paper because they make the standard concrete. A repository cannot be agent-ready if the common toolchain escape hatches are treated as harmless style.

A. Rust

Rust improves the default safety baseline, but its escape hatches are deliberately sharp. Undocumented `unsafe`, public `unsafe fn` without caller obligations, raw ownership reconstruction, unchecked indexing, fabricated UTF-8, broad lint suppression, and dynamic shell execution all move correctness out of the compiler and into a proof obligation. In vibe coding, the recurring failure is to silence the borrow checker or type checker until the program compiles, then treat compilation as evidence. That is not defensible. The reviewable form is local: every unsafe assumption needs a nearby `SAFETY:` argument or equivalent contract, tests must exercise the invalid case where possible, and shell or FFI boundaries need security proof rather than a verbal explanation.

B. SQL and PostgreSQL

SQL failures become durable quickly because the database remembers them. String-built queries turn input into code. Destructive migrations can remove data faster than reviewers can reconstruct intent. Missing constraints outsource truth to every caller forever. PostgreSQL-specific behavior also matters: null semantics, locking, migration order, isolation levels, indexes, and dialect details are not generic SQL trivia. A generated migration that works on an empty local database is not merge evidence. Jankurai expects parameter binding or identifier allowlists, database constraints for durable invariants, migration receipts with rollback or forward-repair paths, row-count and lock evidence for large tables, and tests for tenant isolation, nulls, duplicates, and concurrent writes.

C. TypeScript

TypeScript makes product surfaces easier to review only when the type boundary is honest. `as any`, double casts, disabled strictness, unchecked `JSON.parse`, raw `response.json()`, raw HTML sinks, shell execution, and raw SQL calls all create a false typed surface. The code looks typed while the runtime boundary remains unproved. That is especially risky in agent-authored UI and API glue because the model can satisfy the compiler by moving uncertainty into casts. The repair is not more type theater. The repair is a runtime decoder or generated contract at the boundary, strict compiler evidence, negative tests for malformed input, and rendered or API receipts showing the changed behavior under realistic failure cases.

D. Python

Python is not bad by identity, but dynamic execution paths are hard to defend in a repository that needs deterministic merge evidence. `eval`, `exec`, unsafe object deserialization, `shell=True`, string-built database access, TLS verification bypasses, and broad exception swallowing all erase the boundary that reviewers need. In this workspace's reference profile, Python is additionally constrained because it must not own product truth, authorization, production database writes, proof lanes, or repo tooling except under a dated advanced-ML/data exception. The reviewable form is a boxed service boundary, typed inputs and outputs, safe loaders, argv-array subprocess calls, parameterized database access, explicit exception ownership, and proof that the Python surface cannot silently become durable truth.

E. Docker

Containers are often treated as disposable build packaging, but Docker files are authority maps. `-privileged`, Docker socket mounts, host namespaces, broad capabilities, unconfined security profiles, mutable tags, public database ports, secret-bearing build layers, and remote install pipes all weaken the isolation reviewers think they have. In vibe-coded infrastructure, these changes often appear as one-line fixes for a failing build. That is not an acceptable proof. A defensible container change pins images by digest or stable version, verifies remote downloads, keeps secrets out of build context and layers, runs as least privilege, limits published ports, and leaves a security or image-scan receipt.

F. CI

CI is the repository's execution boundary. Bad CI behavior is therefore not merely operational debt; it can turn a pull request into privileged code execution. The dangerous patterns are well known: `pull_request_target` combined with untrusted checkout, broad `write-all` permissions, mutable action references, secret printing, artifacts or caches that include credentials, privileged Docker on untrusted runners, and security scans configured to continue on error. An agent that can edit CI can also edit the proof system unless the repository separates trusted and untrusted contexts. Jankurai expects minimum permissions, pinned actions, blocking security jobs, workflow owner review, artifact provenance, CI diff receipts, and explicit trust separation before a CI change can support a merge.

G. Git and Git Tooling

Git is safe when history and working-tree state are explicit. It is dangerous when automation hides state or mutates too broadly. Hard resets, destructive cleans, stash-based workflows, force pushes, broad `git add .`, `-no-verify`, destructive ref edits, and credential-bearing remotes all defeat review locality. Hooks, MCP servers, skills, terminal agents, and IDE agents add the same risk through another door: they can mutate repository state while looking like helper infrastructure. A reviewable Git tool names exact paths, records status before staging, avoids hidden state, refuses verification bypass, scopes destructive operations to explicit temporary paths, and leaves tool identity plus changed-path receipts.

Across these families, the pattern is stable. A bad behavior is non-defensible when it moves trust from a machine-checkable boundary into an unrecorded assumption. A Jankurai repair makes the assumption local, typed, constrained, tested, and receipted. That is why the rule IDs matter: `HLT-029` through `HLT-035` are not language rankings. They are merge-evidence obligations for the places where vibe coding most often converts speed into hidden risk.

XX. Related Work

Programming history supplies the first cluster. Structured programming, information hiding, complexity measurement, Conway's law, software evolution, and theory building all explain why code is hard to change safely even when syntax improves. Jankurai does not compete with those ideas. It asks the repository to preserve their operational consequences as owner routes, boundaries, proof lanes, and repair receipts.

Technical-debt and refactoring work supplies the second cluster. Debt is not only messy code; it is accumulated interest on decisions whose consequences are no longer local, visible, or cheaply repairable. In AI-assisted repositories, missing evidence has the same shape: a change may be syntactically clean while still lacking proof of ownership, behavior, generated-source provenance, or waiver repayment. Jankurai's repair queue is a debt-management interface for merge evidence.

AI coding benchmarks and agents measure task completion, environment use, resource limits, context retrieval, and agent-computer interfaces. Supply-chain standards secure artifacts and dependencies. API/schema standards define contract surfaces. Conformance ecosystems show how standards become adoptable. Agent instruction surfaces are emerging as repository policy. UX, accessibility, observability, SARIF, and JUnit ecosystems provide receipts. ML operations work also warns that automation can carry hidden technical debt when boundaries, data dependencies, and feedback loops are implicit [83].

Jankurai's contribution is the missing merge-time binding layer. It does not replace CI, CODEOWNERS-style review, SLSA/SPDX provenance, OpenAPI, Playwright, SARIF, JUnit, OpenTelemetry, agent instruction files, or technical-debt practice. It asks those systems to leave compatible evidence, then binds their outputs to changed paths, owners, proof lanes, missing evidence, repair queues, waiver state, and a closed merge-witness decision. The claim is therefore narrower and more practical than a new universal verifier: existing tools produce signals; Jankurai defines the repository-local object that decides whether those signals are sufficient to merge.

XXI. Limitations and Research Agenda

Jankurai does not prove full semantic correctness. A merge witness can show that changed paths were routed to owners, required proof lanes ran, receipts exist, and missing evidence was handled; it cannot prove every business invariant or exclude malicious compiler, runtime, or organizational governance compromise. Current evidence is conformance-oriented and repository-local. The reference implementation is young, some detectors are advisory or partial, and signed receipts, transparent evidence logs, cross-repo federation, and organizational trust remain future work.

A. Threats to Validity

The taxonomy may reflect source-material bias. Detectors may have false positives and false negatives. Partial rows can be overinterpreted as stronger coverage than they deserve. Reference-profile scoring can bias readers toward Rust/TypeScript/PostgreSQL even when another stack can conform. Browser evidence is environment-sensitive. Screenshots, logs, and traces can leak secrets if redaction fails. Scores can be gamed. Compliance theater can produce artifacts without improving engineering. CI cost can become excessive. Small repositories may need a thinner profile than the paper emphasizes.

B. Failure Modes of Jankurai Itself

Jankurai can fail as artifact theater: reports exist but nobody trusts or uses them. Maps can go stale. Receipts can be faked or copied. Proof lanes can become flaky. Adapters can drift. Reference-profile guidance can harden into stack dogma. Ceremony can overload small changes. These are not reasons to abandon conformance; they are reasons to keep proof freshness, receipt validity, waiver expiry, seed fixtures, and independent implementation tests central.

Important research questions remain open. Which rules best predict escaped defects rather than reviewer discomfort? How much proof is enough for small changes? Which evidence classes can be public without leaking sensitive data? How should registries prevent badge gaming? Which repair queues actually reduce time-to-fix? Jankurai's mitigation strategy is to keep claims versioned, evidence local, waivers expiring, profile guidance non-normative, and independent implementations testable against fixtures.

The strongest version of the criticism is useful: a standard can become ceremony, and a reference profile can become dogma. Jankurai should be judged by whether it improves repair locality, reduces unsafe merges, shortens review cycles, reduces vibe-artifact density, and produces better evidence. If a team can prove those outcomes with a different stack and a compatible control plane, the profile should be documented and studied.

TABLE XLVI
ADJACENT ECOSYSTEMS AND JANKURAI'S BOUNDARY.

Area	Representative work	Jankurai distinction
Programming structure and modularity	Structured programming, modular decomposition, complexity, Conway's law, software evolution, and theory building [5]–[8], [13], [79].	Treats those lessons as repository-evidence obligations for agent-era merge, not only design advice.
Technical debt and refactoring	Technical debt, software aging, debt theory, debt mapping, debt management, and empirical refactoring practice [11], [12], [14]–[17].	Recasts missing owner/proof/receipt evidence as evidence debt and binds repayment to merge witnesses.
Agent benchmarks and agents	SWE-bench, SWE-agent, OpenHands, Aider, SetupBench, ContextBench [31], [32], [40], [47]–[49].	Standardizes the repository-local merge decision those agents must satisfy.
Supply-chain standards	SLSA, SPDX, OSV, OpenSSF Scorecard [24], [28], [63], [64].	Uses evidence mechanics at the pull-request boundary: changed paths, owners, receipts, missing evidence, and witness decisions.
API and schema standards	OpenAPI, OpenAPI Generator, Protocol Buffers/Buf, JSON Schema [25], [73], [74], [80], [81].	Treats contracts as inputs to merge conformance rather than as the whole standard.
Conformance ecosystems	W3C process and Kubernetes conformance [26], [27].	Applies maturity states, levels, fixtures, and submitted evidence to repository merge.
Agent instruction surfaces	`AGENTS.md`, Codex guidance, Claude memory, Cursor rules, Copilot instructions [41]–[45].	Routes instruction surfaces through one conformance model so adapters cannot drift from proof lanes.
UX and observability evidence	Playwright, Storybook, WCAG, OpenTelemetry, JUnit, SARIF [50], [52], [54], [55], [60], [75], [82].	Binds disconnected logs, screenshots, traces, and findings into evidence bundles and merge witnesses.
Repository ownership and policy	CODEOWNERS-style review, policy-as-code, reproducible build tools.	Combines ownership, proof routing, generated zones, permissions, receipts, and repair queues as one versioned repository interface.

TABLE XLVII
CONCRETE FUTURE WORK FOR JANKURAI.

Lane	Expected artifact
Public conformance suite	Runnable tests, fixtures, expected JSON/Markdown reports, and versioned pass criteria.
Hosted or static evidence registry	Immutable evidence bundles, badges, score history, and current/previous standard metadata.
Signed proof receipts and attestations	Artifact digests, command receipts, provenance, and verification command.
Benchmark corpus for repair locality	Repos, patches, route plans, expected lanes, repair time, and escaped-defect labels.
Token-budget optimizer	Short root routers, path-scoped policy packs, symbol/repo maps, filtered command receipts, generated context packs, evidence indexes, stale-doc pruning, and deterministic proof routing.
Context-pack minimizer	Trust-labeled packs with measured token cost, recall, and false-exclusion rates.
Cross-agent adapter conformance tests	Codex, Claude, Cursor, Copilot, terminal-agent, hook, and MCP adapter fixtures.
UI geometry oracle hardening	Viewport matrices, screenshot crops, ARIA snapshots, layout rules, and false-positive corpus.
Semantic rule detector expansion	Fixtures and negative tests for authz, input boundaries, cost controls, release readiness, and human-review evidence.
Waiver and exception libraries	Runtime exception templates plus standards-waiver templates with docs links and repair hints.
Cost-budget and kill-switch tooling	Spend limits, quotas, stop conditions, alerts, and receipt-backed emergency controls.
Reusable cell registry and certification	Cell manifests, install plans, proof lanes, benchmark receipts, and certification policy.

XXII. CONCLUSION

Programming history improved expression, abstraction, runtime safety, and ecosystem reach. It did not solve repository intent. The hard questions remain social and technical at once: who owns this behavior, which boundary is authoritative, what proof is fresh, which waiver is still valid, and whether a refactor changed behavior.

AI makes ambiguity cheaper to create. It can write plausible code in the local dialect faster than a team can review by memory. That does not make agents the enemy; it makes agent-first repository design necessary. Ownership, generated boundaries, proof lanes, stable errors, repair receipts, and merge witnesses must be explicit enough that humans and agents work against the same evidence surface.

The durable contribution is not a stack ranking. It is continuous repository alignment: changed paths, owner route, required proof, observed evidence, missing-evidence decision, artifact digests, commit identity, tool identity, waiver state, and repair queue bound into a merge witness. Generation is cheap; reproducible merge evidence is the new standard. No proof, no merge. No receipt, no trust.

TABLE XLVIII
COMPACT RULE METADATA REGISTRY.

Rule	Family	Required at	Fixture status	Receipt type	Detector maturity
HLT-001	Dead markers / entropy	HL2	detector-backed	fast/audit	deterministic
HLT-002	Generated mutation	HL2	seeded fail	contract	deterministic
HLT-003	Ownerless path	HL2	seeded fail	audit	deterministic
HLT-004	Unmapped proof	HL2	seeded fail	proof plan	deterministic
HLT-005	Product truth in wrong boundary	HL3	mapped	owner/db	heuristic
HLT-006	Direct DB wrong layer	HL3	mapped	db	heuristic
HLT-007	Handwritten contract	HL3	mapped	contract	heuristic
HLT-008	False-green risk	HL3	mapped	fast/release	evidence-routed
HLT-009	Generated security	HL3	mapped	security	evidence-routed
HLT-010	Secret sprawl	HL2	seeded fail	security	deterministic
HLT-011	Prompt injection	HL3	mapped	security	heuristic
HLT-012	Overbroad agency	HL2	seeded fail	permission	heuristic
HLT-013	Rendered UX gap	HL3	seeded fail	web/ux	evidence-routed
HLT-014	Accessibility gap	HL3	mapped	web/a11y	evidence-routed
HLT-015	Context setup gap	HL2	mapped	fast/context	heuristic
HLT-016	Supply-chain drift	HL3	mapped	security	evidence-routed
HLT-017	Opaque observability	HL3	mapped	observability	evidence-routed
HLT-018	Perf/concurrency drift	HL4	mapped	release	evidence-routed
HLT-019	Streaming runtime drift	HL3	mapped	contract/obs.	heuristic
HLT-020	CI hardening gap	HL3	mapped	security/ci	heuristic
HLT-021	Destructive migration	HL2	seeded fail	db	deterministic
HLT-022	Authz isolation gap	HL3	seeded fail	db/security	evidence-routed
HLT-023	Input boundary gap	HL3	seeded fail	security	evidence-routed
HLT-024	Agent-tool supply gap	HL3	mapped	security	governance-backed
HLT-025	Release readiness gap	HL5	mapped	release	governance-backed
HLT-026	Cost budget gap	HL4	mapped	budget	governance-backed
HLT-027	Human-review evidence gap	HL3	mapped	review	evidence-routed
HLT-028	Boundary evidence gap	HL3	mapped	owner/contract	evidence-routed
HLT-029	Rust bad behavior	HL3	detector-backed	fast/security	deterministic
HLT-030	SQL bad behavior	HL3	detector-backed	db	deterministic
HLT-031	TypeScript bad behavior	HL3	detector-backed	fast/security	deterministic
HLT-032	Docker bad behavior	HL3	detector-backed	security	deterministic
HLT-033	Python bad behavior	HL3	detector-backed	contract/security	deterministic
HLT-034	CI bad behavior	HL3	detector-backed	security/ci	deterministic
HLT-035	Git bad behavior	HL3	detector-backed	audit/security	deterministic

APPENDIX A
RULE IDS AND CONFORMANCE EVIDENCE

Stable rule identifiers make audit output durable enough for repair agents, CI annotations, release notes, and independent conformance tests. The compact registry below is the public shape; detailed row coverage remains in `agent/vibe-coverage.toml` and generated reports.

A. Vibe-Coding Source Coverage

Jankurai 0.7.0 maps every parsed row in the vibe-coding corpus into `agent/vibe-coverage.toml`. Coverage is detector-backed only when a narrowed row has deterministic rule coverage plus proof/report evidence; partial when Jankurai has a reviewed control family but still needs issue-specific fixture, CI, or governance receipt evidence. The full per-row registry is supplemental source material in `agent/vibe-coverage.toml`; generated machine-readable outputs are emitted under `target/jankurai/` by the documented coverage command.

APPENDIX B
VERSIONED ARTIFACT MANIFEST

The canonical manifest is `agent/standard-version.toml`. It binds the paper source, rendered paper, agent Markdown companion, full coding standard, and agent standard brief. `target_stack` is the current manifest's profile label; emitted conformance/report/profile claims use `target_stack_id`.

```
standard = "jankurai"
standard_version = "0.7.0"
paper_edition = "2026.05-ed7"
auditor_version = "0.7.0"
```

TABLE XLIX
REPRESENTATIVE RULE REPAIRS.

Rule ID	Required evidence	Default repair
HLT-002-GENERATED-MUTATION	Generated path, source path, regeneration command, diff context.	Change the source contract or generator and regenerate.
HLT-003-OWNERLESS-PATH	Changed path with no owner-map match.	Add owner-map entry or move code into owned path.
HLT-004-UNMAPPED-PROOF	Changed path with no test-map lane.	Add or select deterministic proof command.
HLT-010-SECRET-SPRAWL	Secret-like value, broad env dump, unsafe fixture, prompt transcript leak, or missing secret scan.	Remove secret, rotate if needed, add scanner evidence and redaction tests.
HLT-012-OVERBROAD-AGENCY	Agent/tool lane allows broad shell, browser, network, migration, delete, or credential actions.	Add least-privilege permission profile and approval gate.
HLT-013-RENDERED-UX-GAP	User-facing UI change lacks screenshot, trace, geometry, baseline, or viewport proof.	Add Storybook/Playwright/geometry evidence for changed surface.
HLT-021-DESTRUCTIVE-MIGRATION	Destructive SQL under migration paths without rollback, backfill, lock, or DB proof evidence.	Add migration safety evidence or replace with a non-destructive transition.
HLT-022-AUTHZ-ISOLATION-GAP	Authorization, RLS, tenant, or role surfaces lack owner/non-owner negative proof.	Add role matrix, RLS, and cross-user denial tests.
HLT-023-INPUT-BOUNDARY-GAP	Unsafe input boundary, dynamic execution, string SQL, shell, fetch, or rendering sink lacks negative proof.	Add schemas, parameterization, allowlists, sandboxing, and exploit fixtures.
HLT-028-BOUNDARY-EVIDENCE-GAP	Runtime boundary reclassification lacks owner, proof-lane, contract, or compatibility evidence.	Keep the boundary stable or add deterministic owner/proof/contract receipts.
HLT-029-RUST-BAD-BEHAVIOR	Unsafe, unchecked, shell, FFI, or lint-suppression shortcuts lack local proof.	Remove the shortcut or add local safety contracts, negative tests, and security proof.
HLT-030-SQL-BAD-BEHAVIOR	SQL strings, destructive migrations, or unscoped writes lack DB proof.	Parameterize, constrain, stage, and add migration safety receipts.
HLT-031-TYPESCRIPT-BAD-BEHAVIOR	TypeScript hides runtime uncertainty behind casts, suppressions, disabled strictness, or dynamic sinks.	Validate at boundaries, restore strictness, and add generated contract or negative-input proof.
HLT-032-DOCKER-BAD-BEHAVIOR	Container builds or compose files weaken isolation, bake secrets, or use mutable/unverified inputs.	Pin, verify, remove privilege, isolate secrets, and attach security scan evidence.
HLT-033-PYTHON-BAD-BEHAVIOR	Python dynamic execution, unsafe deserialization, shell, DB, TLS, or product-truth paths lack exception proof.	Box Python behind a typed approved boundary or move behavior to the owning layer.
HLT-034-CI-BAD-BEHAVIOR	CI runs untrusted privileged code, leaks secrets, weakens permissions, or makes security nonblocking.	Split trust contexts, pin actions, minimize scopes, and make security proof blocking.
HLT-035-GIT-BAD-BEHAVIOR	Git automation hides state, stages broadly, bypasses checks, or mutates refs destructively.	Use explicit paths, status receipts, reviewed ref operations, and verification-on workflows.

TABLE L
COMPACT VIBE-COVERAGE REGISTRY SUMMARY FOR V0.7.0.

Status or family	Count	Interpretation
Source rows mapped	260	Every parsed row in the corpus has one canonical issue.
Detector-backed	17	Narrow issue families with deterministic detector evidence and audit/report artifacts.
Partial	243	Reviewed control families that still require issue-specific fixture, CI, or governance evidence.
Unmapped / none	0	No source rows are intentionally left without a control-family mapping in this release.
Security TLR rows	117	Largest source bucket; includes secrets, prompt injection, agency, supply chain, and input/security boundaries.
Verification TLR rows	61	Includes false-green tests, UI/a11y proof, and destructive/release checks that route through proof lanes.
Business-truth TLR rows	24	Includes authorization, data isolation, and state-machine correctness risks.

```
schema_version = "1.5.0"
target_stack = "rust-ts-vite-react-postgres-bounded-python"
published = "2026-05-04"
release_tag = "v0.7.0"
supersedes = "v0.6.x"
compatibility = "schema minor-compatible"
license = "project license"

[[artifact]]
id = "paper-source"
path = "paper/jankurai.tex"
kind = "tex-wrapper"
version_field = "paper_edition"
schema_path = "n/a"
source = "paper/tex/"
command = "just paper"
generated = false
canonical = true
sha256 = "sha256:..."

[[artifact]]
id = "paper-render"
path = "paper/jankurai.pdf"
```

```
kind = "pdf"
version_field = "paper_edition"
source = "paper/jankurai.tex"
command = "just paper"
generated = true
canonical = false
sha256 = "sha256:..."
```

`cargo run -p jankurai -- versions` verifies that `VERSION`, `crates/jankurai/Cargo.toml`, CLI constants, standard documents, companion Markdown, artifact bindings, and generated-zone registration agree.

Appendix C
Waiver and Repair Templates

Waivers are allowed only when they are named, scoped, owned, testable, and temporary enough for an agent to repair later. Runtime exceptions are program errors; standards deviations are waivers.

```
---
id: WVR-YYYY-NNN
state: proposed | active | renewed | expired | closed | superseded
owner: <team or owner cell>
scope:
  - <exact file, crate, service, contract, or zone>
standard_version: "0.7.0"
auditor_version: "0.7.0"
schema_version: "1.5.0"
reason: <why the normal standard cannot be followed now>
compensating_control: <alternate evidence or manual gate>
required_proof:
  - <test-map lane name and command>
residual_risk: <bounded risk statement>
expires_at: <date or measurable exit condition>
owner_approval: <receipt or reviewer>
---

# WVR-YYYY-NNN: <short name>

## Repair path
- <first bounded repair pattern>
- <second bounded repair pattern>
```

```
---
receipt_id: RPR-YYYY-NNN
standard_version: "0.7.0"
auditor_version: "0.7.0"
schema_version: "1.5.0"
before_commit: <sha>
after_commit: <sha>
started_at: <timestamp>
completed_at: <timestamp>
command_digest: sha256:...
log_digest: sha256:...
artifact_digests:
  agent/repo-score.json: sha256:...
decision: pass | review | block | ratchet_fail | release_fail
waiver_closed: <id or false>
residual_risk: <none or bounded risk>
redaction_status: clean | redacted | restricted
owner_approval: <receipt or reviewer>
expires_at: <required if partial>
---

# Repair receipt

Changed paths, proof lane, result, and next repair.
```

Appendix D
Reference-Profile File Tree Diagrams

The body uses short diagrams so the reader can see the intended shape without leaving the conformance argument. The expanded diagrams below are the reference-profile target shape for one default Jankurai repository. Names may vary, but ownership, proof lanes, generated zones, durable truth, and repair evidence should remain explicit.

```
repo/
  [PROFILE] apps/
    web/                        TypeScript/React/Vite UI
      src/
        app/                    routes, shells, providers
        components/             design-system composition only
        generated/              [GENERATED] API clients/contracts
```

```
      stories/                 Storybook state matrix
    playwright/                [EVIDENCE] critical flows/screens
  api/                         Rust HTTP/RPC edge
[PROFILE] crates/
  domain/                      pure invariants and policy
  application/                 use cases, authz, transactions
  adapters/                    DB, queues, external APIs
  workers/                     async jobs and projections
[CORE] contracts/
  openapi/                     OpenAPI source
  proto/                       protobuf source
  jsonschema/                  emitted JSON Schema when needed
[PROFILE] db/
  migrations/                  durable schema history
[PROFILE] python/ai-service/   bounded AI/data boundary only
[EVIDENCE] packages/ux-qa/     rendered UX geometry runtime
```

```
repo/
  [CORE] AGENTS.md               short root router
  [CORE] agent/
    JANKURAI_STANDARD.md         brief agent bootstrap
    owner-map.json               path -> accountable owner
    test-map.json                path -> proof lane
    proof-lanes.toml             runnable validation commands
    generated-zones.toml         generated file policy
    standard-version.toml        versioned artifact manifest
    audit-policy.toml            local scoring and waivers
  [CORE] docs/
    agent-native-standard.md     full standard
    mission.md                   paper mission and scope
    testing.md                   lane doctrine
    waivers/                     named temporary deviations
    repair/                      repair receipts and artifacts
  [EVIDENCE] agent/repo-score.json
  [EVIDENCE] agent/repo-score.md
  [EVIDENCE] target/jankurai/
    merge-witness.json
    evidence-index.json
    receipts/
```

APPENDIX E
GOLDEN FIRST-HOUR COMMAND PATH

The command map below is operational documentation for agents. It is not a full CLI manual and not a claim that every repository should enable every lane on day one.

```
jankurai adopt . --profile auto --mode observe \
  --out target/jankurai/adoption-plan.json \
  --md target/jankurai/adoption-plan.md

jankurai init . --level agents --dry-run \
  --plan-json target/jankurai/init-agents.json
jankurai init . --level agents --yes

jankurai init . --level score --yes
jankurai audit . --mode advisory \
  --json target/jankurai/repo-score.json \
  --md target/jankurai/repo-score.md

jankurai lane . --changed README.md \
  --out target/jankurai/proof-plan.json \
  --md target/jankurai/proof-plan.md

jankurai prove . --changed README.md \
  --plan-out target/jankurai/proof-plan.json \
  --plan-md target/jankurai/proof-plan.md

jankurai witness . --changed-from origin/main \
  --baseline agent/repo-score.json \
  --out target/jankurai/merge-witness.json \
  --md target/jankurai/merge-witness.md
```

A. Failure Example

```
jankurai witness . --changed-from origin/main \
  --baseline agent/repo-score.json \
  --out target/jankurai/merge-witness.json

# Expected decision when a changed generated file lacks
# source regeneration proof:
# decision = "block"
```

```
# missing_evidence = ["contract_receipt"]
# repair_queue[0].rule_id = "HLT-002-GENERATED-MUTATION"
```

APPENDIX F
PUBLIC REPOSITORY SCORE DETAILS

The table below is generated from `paper/data/public-repo-scores-20260505T184426Z.json`. The adjacent receipt file records the SHA-256 digest for the tracked source JSON. Rank is the generated score order; run number preserves the original input order from the May 5, 2026 advisory run.

TABLE LI: Full 30-repository advisory scoring run.

Rank	Run #	Repo	Score	Findings	Hard	Dominant gaps
1	25	zed-industries/zed	**48** best observed	2,029	2,016	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
2	26	astral-sh/ruff	**45**	2,752	2,739	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
3	23	denoland/deno	**43**	1,615	1,601	code shape/semantic surface; audit CI replacement; context routing
4	6	RightNow-AI/openfang	**42**	690	677	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
5	29	meilisearch/meilisearch	**42**	696	685	code shape/semantic surface; audit CI replacement; proof routing
6	9	nearai/ironclaw	**42**	1,798	1,785	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
7	4	googleworkspace/cli	**41**	97	86	code shape/semantic surface; audit CI replacement; proof routing
8	3	zeroclaw-labs/zeroclaw	**40**	1,213	1,200	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
9	12	kvnxiao/tauri-tanstack-start-react-template	**39**	22	10	audit CI replacement; context routing; proof routing
10	2	vercel-labs/agent-browser	**38**	189	178	code shape/semantic surface; audit CI replacement; security/supply chain
11	21	tauri-apps/tauri	**38**	304	292	code shape/semantic surface; audit CI replacement; context routing
12	24	astral-sh/uv	**38**	1,140	1,128	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
13	19	kevinpiao1025/tauri-react-typescript-tailwind	**36**	22	10	audit CI replacement; proof routing; context routing
14	5	AlexsJones/llmfit	**35**	107	94	code shape/semantic surface; Python/polyglot hygiene; audit CI replacement
15	27	alacritty/alacritty	**34**	110	98	code shape/semantic surface; context routing; audit CI replacement
16	16	exit-zero-labs/threat-forge	**32**	134	121	code shape/semantic surface; audit CI replacement; context routing
17	8	microsoft/RustTraining	**31**	34	21	code shape/semantic surface; audit CI replacement; context routing
18	28	BurntSushi/ripgrep	**31**	72	60	code shape/semantic surface; audit CI replacement; context routing
19	11	octasoft-ltd/wsl-ui	**31**	314	302	code shape/semantic surface; audit CI replacement; context routing
20	30	typst/typst	**31**	382	370	code shape/semantic surface; audit CI replacement; context routing
21	22	rustdesk/rustdesk	**30**	648	636	code shape/semantic surface; proof routing; Python/polyglot hygiene
22	1	rtk-ai/rtk	**28**	397	383	code shape/semantic surface; audit CI replacement; context routing
23	20	MarkShawn2020/lovtauri	**26**	30	18	audit CI replacement; proof routing; code shape/semantic surface
24	18	Duri686/RustQuantLab	**26**	42	30	code shape/semantic surface; audit CI replacement; context routing
25	10	h4ckf0r0day/obscura	**26**	60	48	code shape/semantic surface; audit CI replacement; proof routing
26	7	xai-org/x-algorithm	**24**	38	25	code shape/semantic surface; audit CI replacement; Python/polyglot hygiene
27	14	sergioadevita/notemac-plus-plus	**24**	293	279	code shape/semantic surface; audit CI replacement; Python/polyglot hygiene
28	17	lostf1sh/rustune	**23**	36	24	code shape/semantic surface; audit CI replacement; proof routing
29	13	fudanglp/tauri-fastapi-full-stack-template	**23**	69	56	code shape/semantic surface; audit CI replacement; Python/polyglot hygiene
30	15	ianho7/maptoposter-online	**14**	58	45	build speed; code shape/semantic surface; audit CI replacement

APPENDIX G
LANGUAGE BAD-BEHAVIOR MATRIX

The language bad-behavior family turns common vibe-coding escape hatches into stable, reviewable obligations. The matrix below is intentionally practical: each row names behavior that should be hard to merge without local evidence, the detector or

rule family that routes it, the receipt that makes it reviewable, and the default repair.

TABLE LII: Language and toolchain bad-behavior matrix.

Surface	Hard reject examples	Rule or detector IDs	Required evidence	Repair default
Runtime boundary	Reclassifying a runtime, service, adapter, or durable-truth boundary without deterministic proof of the new owner and proof lane.	HLT-028-BOUNDARY-EVIDENCE-GAP; owner/proof map audit	Owner map, test map, generated-zone status, contract evidence, and migration or compatibility receipt for the changed boundary.	Keep the boundary unchanged or add the missing owner/proof map and compatibility evidence.
Rust	Undocumented `unsafe`, public `unsafe fn` without a `# Safety` contract, raw ownership reconstruction, unchecked indexing, `static mut`, broad lint suppression, dynamic shell execution.	HLT-029-RUST-BAD-BEHAVIOR; rust.unsafe.*; rust.security.shell-c-dynamic	Nearby `SAFETY:` proof, caller-obligation docs, focused negative tests, security proof for shell/FFI paths.	Remove the escape hatch or make the invariant local, documented, and tested.
SQL/PostgreSQL	Dynamic SQL strings, `DROP`/`TRUNCATE`/`DROP COLUMN` without safety evidence, `UPDATE`/`DELETE` without scope, schema truth enforced only by app code.	HLT-030-SQL-BAD-BEHAVIOR; sql.dynamic-sql; sql.migration.destructive-command; HLT-021	Parameter binding, identifier allowlist, migration rollback or forward repair, row-count and lock plan, DB proof lane.	Parameterize, add constraints, split or stage the migration, and document recovery.
TypeScript	`@ts-nocheck`, `@ts-ignore`, `as any`, double casts, unchecked JSON or request bodies, disabled strictness, raw HTML, shell, or SQL sinks.	HLT-031-TYPESCRIPT-BAD-BEHAVIOR; typescript.suppress.*; typescript.types.any-boundary; typescript.runtime.*	Strict compiler proof, runtime decoder/schema, generated contract evidence, negative tests for malformed input.	Restore strictness, validate before narrowing, and replace dynamic sinks with typed or sanitized paths.
Docker	Docker socket mounts, `-privileged`, host namespaces, dangerous capabilities, unconfined profiles, secrets in layers, remote install pipes, mutable tags.	HLT-032-DOCKER-BAD-BEHAVIOR; docker.compose.*; docker.install.unverified-remote; docker.image.mutable-tag	Least-privilege profile, pinned digest or version, checksum/signature evidence, secret-free context, image/security scan.	Remove privilege, pin and verify inputs, move secrets to runtime, and narrow exposed ports.
Python	`eval`, `exec`, unsafe deserialization, `shell=True`, string-built DB execution, TLS verification bypass, broad exception handling, unapproved product-truth ownership.	HLT-033-PYTHON-BAD-BEHAVIOR; python.exec.dynamic-code; python.deser.*; python.shell.dynamic; HLT-005	Dated exception when needed, typed service boundary, safe loader, argv-array subprocess call, parameterized DB access, containment tests.	Replace dynamic execution, box the service, move truth to approved layers, and add boundary proof.
CI	`pull_request_target` executing PR head, `write-all`, secret printing, nonblocking security scans, mutable actions, privileged Docker on untrusted runners, credential caches.	HLT-034-CI-BAD-BEHAVIOR; ci.github.*; ci.permissions.write-all; ci.security-scan.nonblocking; HLT-020	Pinned actions, least-privilege permissions, trusted/untrusted separation, blocking security jobs, artifact provenance, workflow owner review.	Split trust contexts, minimize token scopes, pin actions, and make security failures blocking.
Git automation	`git reset --hard`, destructive clean, stash hidden state, force push, broad staging, `--no-verify`, destructive ref edits, credential-bearing remotes.	HLT-035-GIT-BAD-BEHAVIOR; git.destructive.*; git.stage.unbounded; git.remote.*	Status receipt, explicit path list, no hidden state, reviewed ref operation, exact staged-path evidence.	Replace broad mutation with path-scoped commands and keep verification enabled.
Git tools, hooks, and agents	Hooks, MCP tools, skills, or IDE/terminal agents with broad mutation authority, unknown provenance, dynamic remote instructions, or no receipts.	HLT-024-AGENT-TOOL-SUPPLY-CHAIN; HLT-035-GIT-BAD-BEHAVIOR; HLT-012	Tool identity, supply-chain review, least-privilege permission profile, changed-path receipt, stop conditions, and rollback plan.	Pin and review the tool, reduce permissions, route mutations through explicit receipts, or disable the hook/agent.

References

[1] J. Backus, "Can programming be liberated from the von Neumann style? a functional style and its algebra of programs," *Communications of the ACM*, vol. 21, no. 8, pp. 613–641, 1978.

[2] D. M. Ritchie, "The development of the C language," in *History of Programming Languages—II*. ACM, 1993, pp. 201–208.

[3] Microsoft, "Announcing TypeScript 7.0 Beta," https://devblogs.microsoft.com/typescript/announcing-typescript-7-0-beta/, 2026, TypeScript Dev Blog, published 2026-04-21; accessed 2026-05-01.

[4] GitHub, "Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1," https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/, 2025, GitHub Blog, published 2025; accessed 2026-05-01.

[5] E. W. Dijkstra, "Go to statement considered harmful," *Communications of the ACM*, vol. 11, no. 3, pp. 147–148, 1968.

[6] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

[7] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.

[8] M. E. Conway, "How do committees invent?" *Datamation*, vol. 14, no. 4, pp. 28–31, 1968.

[9] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.

[10] F. P. Brooks, "No silver bullet: Essence and accidents of software engineering," *Computer*, vol. 20, no. 4, pp. 10–19, 1987.

[11] W. Cunningham, "The wycash portfolio management system," in *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications*. ACM, 1992, pp. 29–30.

[12] D. L. Parnas, "Software aging," in *Proceedings of the 16th International Conference on Software Engineering*. IEEE Computer Society, 1994, pp. 279–287.

[13] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proceedings of the IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980.

[14] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012.

[15] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015.

[16] N. Brown, Y. Cai, Y. Guo, R. Kazman, M. Kim, P. Kruchten, E. Lim, A. MacCormack, R. Nord, I. Ozkaya, R. Sangwan, C. Seaman, K. Sullivan, and N. Zazworka, "Managing technical debt in software-reliant systems," in *Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research*. ACM, 2010, pp. 47–52.

[17] E. Murphy-Hill, C. Parnin, and A. P. Black, "How we refactor, and how we know it," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 5–18, 2012.

[18] DORA, "State of AI-assisted Software Development 2025," https://dora.dev/research/2025/dora-report/, 2025, DORA research page, accessed 2026-05-01.

[19] Veracode, "2025 GenAI Code Security Report," https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf, 2025, official report PDF, accessed 2026-05-01.

[20] GitGuardian, "The state of secrets sprawl report 2026," https://www.gitguardian.com/state-of-secrets-sprawl-report-2026, 2026, annual report page, accessed 2026-05-01.

[21] ——, "The State of Secrets Sprawl 2026: AI-Service Leaks Surge 81% and 29M Secrets Hit Public GitHub," https://blog.gitguardian.com/the-state-of-secrets-sprawl-2026/, 2026, official companion blog summary, published 2026-03-17; accessed 2026-05-01.

[22] RFC Editor, "Best Current Practice 14," https://www.rfc-editor.org/info/bcp14, 2026, RFC 2119 and RFC 8174 key words for requirements; accessed 2026-05-04.

[23] Semantic Versioning contributors, "Semantic versioning 2.0.0," https://semver.org/, 2026, official specification; accessed 2026-05-04.

[24] SPDX Project, "SPDX Specifications," https://spdx.dev/use/specifications/, 2026, official current and previous specification page; accessed 2026-05-04.

[25] OpenAPI Initiative, "OpenAPI Specification," https://spec.openapis.org/oas/, 2026, official specification site with schema guidance; accessed 2026-05-04.

[26] World Wide Web Consortium, "W3C process document," https://www.w3.org/policies/process/, 2026, official process document; accessed 2026-05-04.

[27] Cloud Native Computing Foundation, "Certified kubernetes conformance program," https://www.cncf.io/training/certification/software-conformance/, 2026, official CNCF software conformance page; accessed 2026-05-04.

[28] Open Source Security Foundation, "OpenSSF scorecard," https://openssf.org/scorecard/, 2026, official project page; accessed 2026-05-04.

[29] Stack Overflow, "AI: 2025 Developer Survey," https://survey.stackoverflow.co/2025/ai, 2025, official developer survey AI section; accessed 2026-05-01.

[30] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications v2025," https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf, 2025, official OWASP project PDF; accessed 2026-05-01.

[31] SetupBench Authors, "Setupbench: Assessing software engineering agents' ability to bootstrap development environments," *arXiv preprint arXiv:2507.09063*, 2025.

[32] ContextBench Authors, "Contextbench: A benchmark for context retrieval in coding agents," *arXiv preprint arXiv:2602.05892*, 2026.

[33] Jankurai Project, "Public repository advisory scoring run: 30 public github repositories," paper/data/public-repo-scores-20260505T184426Z.json, 2026, generated 2026-05-05T18:45:33Z with Jankurai 0.7.0; run root `/home/ubuntu/jankscore/runs/20260505T184426Z`; receipt paper/data/public-repo-scores-20260505T184426Z.json.sha256.

[34] Zed Industries, "zed-industries/zed," https://github.com/zed-industries/zed, 2026, public GitHub repository; accessed 2026-05-05.

[35] Astral, "astral-sh/ruff," https://github.com/astral-sh/ruff, 2026, public GitHub repository; accessed 2026-05-05.

[36] Deno Land, "denoland/deno," https://github.com/denoland/deno, 2026, public GitHub repository; accessed 2026-05-05.

[37] Meilisearch, "meilisearch/meilisearch," https://github.com/meilisearch/meilisearch, 2026, public GitHub repository; accessed 2026-05-05.

[38] Tauri Programme within The Commons Conservancy, "tauri-apps/tauri," https://github.com/tauri-apps/tauri, 2026, public GitHub repository; accessed 2026-05-05.

[39] ripgrep contributors, "BurntSushi/ripgrep," https://github.com/BurntSushi/ripgrep, 2026, public GitHub repository; accessed 2026-05-05.

[40] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024.

[41] OpenAI, "Custom instructions with AGENTS.md," https://developers.openai.com/codex/guides/agents-md, 2026, codex documentation, accessed 2026-05-01.

[42] AGENTS.md contributors, "AGENTS.md," https://agents.md/, 2026, agent instruction file specification and guidance; accessed 2026-05-01.

[43] Anthropic, "How claude remembers your project," https://code.claude.com/docs/en/memory, 2026, claude Code documentation, accessed 2026-05-01.

[44] Cursor, "Cursor rules documentation," https://docs.cursor.com/en/context, 2026, cursor documentation, accessed 2026-05-01.

[45] GitHub, "Adding repository custom instructions for GitHub Copilot," https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions, 2026, official documentation, accessed 2026-05-01.

[46] SWE-Effi Authors, "SWE-Effi: Re-Evaluating Software AI Agent System Effectiveness Under Resource Constraints," *arXiv preprint arXiv:2509.09853*, 2025.

[47] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *International Conference on Learning Representations*, 2024.

[48] OpenHands Authors, "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," *arXiv preprint arXiv:2407.16741*, 2024.

[49] Aider Contributors, "Aider: AI pair programming in your terminal," https://github.com/Aider-AI/aider, 2026, GitHub repository, accessed 2026-05-01.

[50] Microsoft Playwright, "Playwright best practices," https://playwright.dev/docs/best-practices, 2026, playwright documentation, accessed 2026-05-01.

[51] ——, “Writing tests,” https://playwright.dev/docs/writing-tests, 2026, official documentation; accessed 2026-05-01.
[52] World Wide Web Consortium, “Understanding success criterion 2.5.8: Target size (minimum),” https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html, 2026, W3C Web Accessibility Initiative documentation; accessed 2026-05-01.
[53] web.dev, “Optimize cumulative layout shift,” https://web.dev/articles/optimize-cls, 2025, last updated 2025-02-07; accessed 2026-05-01.
[54] Storybook, “Visual tests,” https://storybook.js.org/docs/9/writing-tests/visual-testing, 2026, official documentation; accessed 2026-05-01.
[55] Microsoft Playwright, “Visual comparisons,” https://playwright.dev/docs/test-snapshots, 2026, official documentation; accessed 2026-05-01.
[56] ——, “Snapshot testing: Aria snapshots,” https://playwright.dev/docs/aria-snapshots, 2026, official documentation; accessed 2026-05-01.
[57] Argos CI, “Argos visual testing,” https://argos-ci.com/visual-testing, 2026, product documentation; accessed 2026-05-01.
[58] BackstopJS contributors, “BackstopJS,” https://github.com/garris/BackstopJS, 2026, open-source repository; accessed 2026-05-01.
[59] Microsoft Playwright, “Accessibility testing,” https://playwright.dev/docs/accessibility-testing, 2026, official documentation; accessed 2026-05-01.
[60] Storybook, “Accessibility tests,” https://storybook.js.org/docs/writing-tests/accessibility-testing, 2026, official documentation; accessed 2026-05-01.
[61] National Security Agency (NSA) and Cybersecurity and Infrastructure Security Agency (CISA), “Memory safe languages: Reducing vulnerabilities in modern software development,” https://www.cisa.gov/resources-tools/resources/memory-safe-languages-reducing-vulnerabilities-modern-software-development, 2025, NSA/CISA cybersecurity information sheet, June 2025; accessed 2026-05-01.
[62] GitHub, “Working with secret scanning and push protection,” https://docs.github.com/en/code-security/secret-scanning/working-with-secret-scanning-and-push-protection, 2026, official documentation; accessed 2026-05-01.
[63] Google OSV, “OSV-Scanner,” https://google.github.io/osv-scanner/, 2026, official documentation; accessed 2026-05-01.
[64] SLSA Framework contributors, “Supply-chain Levels for Software Artifacts,” https://slsa.dev/, 2026, official specification site; accessed 2026-05-01.
[65] M. Nottingham, E. Wilde, and S. Dalal, “Problem Details for HTTP APIs,” https://www.rfc-editor.org/rfc/rfc9457, 2023, RFC 9457.
[66] OpenTelemetry Authors, “OpenTelemetry Semantic Conventions,” https://opentelemetry.io/docs/specs/otel/semantic-conventions/, 2026, official documentation, accessed 2026-05-01.
[67] ——, “Exception attributes,” https://opentelemetry.io/docs/specs/semconv/registry/attributes/exception/, 2026, official semantic conventions; accessed 2026-05-01.
[68] MDN Web Docs contributors, “MDN error: cause,” https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/cause, 2026, MDN Web Docs; accessed 2026-05-01.
[69] Rust Project, “Rust error handling,” https://doc.rust-lang.org/stable/rust-by-example/error.html, 2026, Rust By Example; accessed 2026-05-01.
[70] anyhow contributors, “anyhow,” https://docs.rs/anyhow/latest/anyhow/, 2026, Rust crate documentation; accessed 2026-05-01.
[71] Python Software Foundation, “Python Release Python 3.14.0,” https://www.python.org/downloads/release/python-3140/, 2025, release date Oct. 7, 2025; accessed 2026-05-01.
[72] The Go Programming Language, “Results from the 2025 go developer survey,” https://go.dev/blog/survey2025, 2026, survey conducted Sept. 9-30, 2025; results published 2026; accessed 2026-05-01.
[73] OpenAPI Generator contributors, “OpenAPI Generator usage,” https://openapi-generator.tech/docs/usage, 2026, official documentation; accessed 2026-05-01.
[74] Buf, “Buf documentation,” https://buf.build/docs/, 2026, official documentation; accessed 2026-05-01.
[75] OpenTelemetry Authors, “OpenTelemetry Documentation,” https://opentelemetry.io/docs/, 2025, official documentation landing page, last modified 2025-08-29; accessed 2026-05-01.
[76] PostgreSQL Global Development Group, “PostgreSQL constraints,” https://www.postgresql.org/docs/current/ddl-constraints.html, 2026, PostgreSQL documentation; accessed 2026-05-01.
[77] ——, “PostgreSQL row security policies,” https://www.postgresql.org/docs/current/ddl-rowsecurity.html, 2026, PostgreSQL documentation; accessed 2026-05-01.
[78] Vite Team, “Vite 8.0 is out!” https://vite.dev/blog/announcing-vite8, 2026, vite blog, published 2026-03-12; accessed 2026-05-01.
[79] P. Naur, “Programming as theory building,” *Microprocessing and Microprogramming*, vol. 15, no. 5, pp. 253–261, 1985.
[80] Protocol Buffers contributors, “Protocol Buffers Documentation,” https://protobuf.dev/overview/, 2026, official documentation; accessed 2026-05-04.
[81] JSON Schema contributors, “JSON Schema Specification,” https://json-schema.org/specification, 2026, official specification page for current and previous drafts; accessed 2026-05-04.
[82] OASIS, “Static Analysis Results Interchange Format (SARIF) Version 2.1.0,” https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html, 2023, OASIS Standard; accessed 2026-05-04.
[83] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, and M. Young, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems 28*, 2015, pp. 2503–2511.